

MSc Data Science Project - 7PAM2002

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Early Warning Signals in League of Legends using
match level statistics

By

KEERTHIVASAN KANNAN | 23052162

Supervisor: **Dr. Stephen Kane**

Date Submitted: 04/01/2026

Word Count: 4882 (excluding references and appendix)

**University of
Hertfordshire UH**

School of Physics,
Engineering and
Computer Science

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in **Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at [Assessment Offences and Academic Misconduct](#) and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: KEERTHIVASAN KANNAN

Student Name signature: Keerthivasan Kannan

Student SRN number: 23052162

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

ABSTRACT

This project talks about whether the bigger changes in statistics in League of Legends can be seen coming around 30 seconds before they happen. I have extracted the match records using **Riot API** to build a detailed timeline that tracks gold flow, vision habits, player positioning and the small changes in game pressure that leads to major objectives. Then I created around 8000 sequences particularly to study the event triggers in the 30 second window. The dataset had class imbalance issue, so I used **jittering** to shift positive samples to mitigate the issue. After creating these sequences, I experimented it with few models, that lead to finding a mix of a hybrid model (Temporal convolutional networks, Bidirectional GRU, cross attention). This model performed at its best even with all the class imbalances where the AUC reached **97%** for baron, **95%** for dragon, and **92%** for teamfights. The results show how the players experiences of sudden twists in the game actually have a pattern that can be studied in advance.

Table of Contents:

DECLARATION STATEMENT	2
ABSTRACT	3
1. INTRODUCTION:.....	7
2. LITERATURE COMPARISON:.....	9
3. METHODOLOGY:	10
3.1 Problem Definition:	10
3.2 Features and preprocessing:	10
3.3 Sequence building and augmentation:.....	10
3.4 Train-validation-test split and dataset:.....	11
3.5 Models I chose to experiment with:	11
3.6 LeagueEWS architecture:	12
3.7 Hyperparameter tuning and training:.....	13
3.8 Evaluation metrics:	15
4. RESULTS:.....	16
4.1 Overall predictive performance:	16
4.2 ROC curves:	16
4.3 Confusion matrices:	17
4.4 Risk plots:	18
5. ETHICAL CONSIDERATIONS:.....	20
6. DISCUSSION:.....	21
6.1 Interpreting the gains:	21
6.2 Improvement of models:.....	21
6.3 Limitations:	21
7. CONCLUSION:.....	22
References	23
Appendix:	24

LIST OF FIGURES:

Figure 1 - Visualization of how teamfight happens in the game..... 7

Figure 2 - Workflow of my project 8

Figure 3 - Multi Layer Perceptron..... 11

Figure 4 - LeagueEWS architecture..... 12

Figure 5 - Epoch plot for LeagueEWS 14

Figure 6 - RF ROC curve..... 14

Figure 7 – Simple MLP ROC curve..... 16

Figure 8 - LeagueEWS ROC curve..... 17

Figure 9 - RF confusion matrix 17

Figure 10 - Simple MLP confusion matrix..... 18

Figure 11 - LeagueEWS confusion matrix..... 18

Figure 12 - RF risk plot..... 19

Figure 13 - Simple MLP risk plot..... 19

Figure 14 - LeagueEWS risk plot 19

LIST OF TABLES:

Table 1 - Comparison of all the literatures I referred9
Table 2 - Target labels 11
Table 3 - Tuning results 13
Table 4 – Evaluation metrics of all models 16

1. INTRODUCTION:

League of Legends is a game that's highly **dynamic**. For an instance, a match feels normal and flip completely the next minute like you see in the Figure 1. As someone who plays ranked matches regularly, I know this feel where my team is leading in major factors hoping for winning the game, suddenly the opposite team completes an objective that gives them a huge advantage. It always made me think whether what triggers this kind of unparallel event. And that lead to give me an idea to apply my technical knowledge to find an answer for it.

There were researchers who already explored about this game using machine learning to understand game outcomes. Cong et al. (2024) pointed out that the previous studies relied on macro level features, that held them back on the accuracy of their prediction models in several MOBO games. (Cong, Wu, Xiong, Wu, & Zuo, 2024). It shows that professional matches have more dense information than simple variables. Hojaji et al. (2025) took a bigger step by applying deep learning to a large dataset and finding which metrics are important at different stages of the match, like turret pressure, vision control etc. (Hojaji, et al., 2025) Both the papers made me realize that League of Legends is a game that surely gives signals during the match if I actually know where to look.



Figure 1 - Visualization of how teamfight happens in the game

With all the observations, knowing I can work on macro level features, I wanted to do something different. Instead of predicting who will win, I tried to have a look at the in-game events. To achieve that, I extracted timeline match records from **2024-2025** season through the **Riot API**. Then I tailored it into a detailed sequence of game every ten seconds. It has factors like gold flow, player movement, pressure around objectives and vision patterns. From this, I tried to forecast whether a Baron take, Dragon kills or large teamfight will occur in a particular time window. These events are extremely rare compared to all macro level events. I met with an issue where the data is too realistic that it had severe class imbalance. So, I used jittering method on positives to mitigate it without affecting the realism of the data.

Initially I tried to look this as a classification problem, so I used **Random Forest**, but it couldn't handle the imbalanced data even after jittering. So, I tried to use an **MLP**, a simple architecture to handle the nonlinear relationships. But it couldn't handle multiple frames where the game changes at short intervals which is quite expected. After experimenting a lot, I built a custom model that mixes **Temporal**

convolutional networks, Bidirectional GRUs and cross attention. Like a binocular, TCN looks for short bursts in the game, while BiGRUs track patterns that will build over time. Cross attention holds this model together by focusing on the significant sequences, filtering out the noise in the gameplay. This model gave me an answer that's close to what I had in my vision, that detects early warning signals of these major events in the game. Figure 2 depicts how my entire workflow looks like



Figure 2 - Workflow of my project

2. LITERATURE COMPARISON:

Table 1 - Comparison of all the literatures I referred

Study	Core Objective	Methodology	Key Difference
Real Time Win Prediction _(Junior & Campelo, 2023)	Predicts the win probability during the gameplay.	Uses static snapshots with fixed intervals and applied on models like LightBGM and MLP as non-sequential, aggregated features.	Compresses time as static samples, so it cannot identify short spikes in game like how I how I'm predicting in a short time window (30 seconds).
Large Scale KPI & Movement Analysis _(Hojaji, et al., 2025)	Identify the most influential metrics that affects the gameplay.	Analyses 154,000 ranked matches and focus on objectives, turrets, gold and map control patterns using statistical and ML methods.	Focuses on macro level features, but not the temporal dynamics or forecasting and sampling is too noisy.
Pregame Win probability Models _(Hitar-García, Morán-Fernández, & Bolón-Canedo, 2023)	Predict the game outcome before game starts.	Creates teamwork based tailored features and apply different models using pregame data.	No temporal data, so there's no way to detect any objective that happens throughout the game.
Pregame Feature Engineered Deep Model _(Cong, Wu, Xiong, Wu, & Zuo, 2024)	Predict the game outcome before game using deep learning architecture.	Uses lineup-based features like champion selection, role interactions and player strength to get high accuracy.	Fully pregame and excludes timeline events, which makes it not suitable for micro scaled probabilities.
Player Performance & Role Importance Analysis _(Bahrololloomi, Klonowski, Sauer, Horst, & Dörner, 2023)	Predict match outcomes using player's behaviour and role-based performance.	Constructs behavioural features and role evaluation scores from multiple previous matches and it trains on classic ML models	Relies on long-time player's performance history instead of evolution in the match dynamics. And there's no refined temporal event features.

3. METHODOLOGY:

3.1 Problem Definition:

The goal of my work is to build an early warning system for League of Legends that predicts the in-game events like **Baron Nashor kill, Dragon kill, Teamfight**. I took three-time horizons (10s/20s/30s) for each of these events, where it gives nine binary prediction tasks in total (3 events x 3 horizons). So, to estimate I looked at each timestep t for a feature x_1

$$P(y_t^{(e,h)} = 1 \mid x_{1:t}),$$

where $e \in \{\text{Baron, Dragon, Teamfight}\}$ and $h \in \{10, 20, 30\}$. While I feed it to the model, the warnings are mostly in 30-second horizon.

3.2 Features and preprocessing:

I treated the dataset like a blackbox to focus on learning setup. I removed the identifiers and labels and calculated difference for certain features between teams (Blue/Red) and ended up with **150 numeric features** that shows the state of the game at time t . I applied two transformations before modelling:

- Per-match feature differences – I want the model look at the chaotic dynamics so for each feature f and each match I computed the first-order difference.

$$f_1^{diff} = f_t - f_{t-1},$$

- Standardisation – I used StandardScalar fit on the whole dataset to z-scale all features, so each column has zero mean and unit variance. It helps the models to converge more comfortably.

At the end of preprocessing, I had 150 input dimensions and 9 labels per time step.

3.3 Sequence building and augmentation:

While I play the game, the chaos happens in a short interval, so to make the model understand these spikes, I converted the tabular data into fixed length sequences using a sliding window. For each match I sorted the rows by time and windows of **40 frames** and moved it forward in **5 steps**. The input is the tensor $X \in \mathbb{R}^{40 \times 150}$. I took the last time step in the window as the label vector. In end, it gave me **74,728 sequences** with shape (40, 150) with 9 target vectors.

$$X^{(s)} = [\tilde{x}_s, \tilde{x}_{s+1}, \dots, \tilde{x}_{s+39}] \in \mathbb{R}^{40 \times 150} \quad | \quad y^{(s)} = y_{s+39},$$

The game stats are too realistic where the positives are extremely low. So, to give the model a power up, I added a simple **jitter augmentation**. I padded the original sequences at the start and end. Then randomly shifted the windows by **± 2 frames**. I repeated this twice, while keeping the labels unchanged. This gives me **224,184 sequences** in the end.

3.4 Train-validation-test split and dataset:

I split the augmented sequences into train (**72%**), validation (**18%**) and test (**10%**) sets. For the neural models, I created TensorFlow datasets where the batch size is 256, used shuffling on train set and enabled Prefetching. Since, there are 3 events, I mapped 9 labels into dictionary for multi-output heads in the model.

Table 2 - Target labels

Baron	y_baron_10, y_baron_20, y_baron_30
Dragon	y_dragon_10, y_dragon_20, y_dragon_30
Teamfight	y_teamfight_10, y_teamfight_20, y_teamfight_30

3.5 Models I chose to experiment with:

After referring to different papers, I wanted to see this first as a classification problem, to witness myself how I can use certain models for my idea. First, I started with Random Forest, where I took the last time step in each sequence as input. Then I concatenated train, and validation sets to maximise training data. For 9 labels, I used nine independent **RandomForestClassifier** for each label. The key hyperparameters are: 120 trees, max depth of 5, each leaf with 5 samples, balanced_subsample as class weight to reduce imbalance and a random state as 42 for reproducibility.

Next, I used a **Simple MLP**, a feed-forward neural network that also uses the last time step. The input are the 150 vectors, 2 hidden layers, output dense layer. I will discuss the results of both models in the further sections.

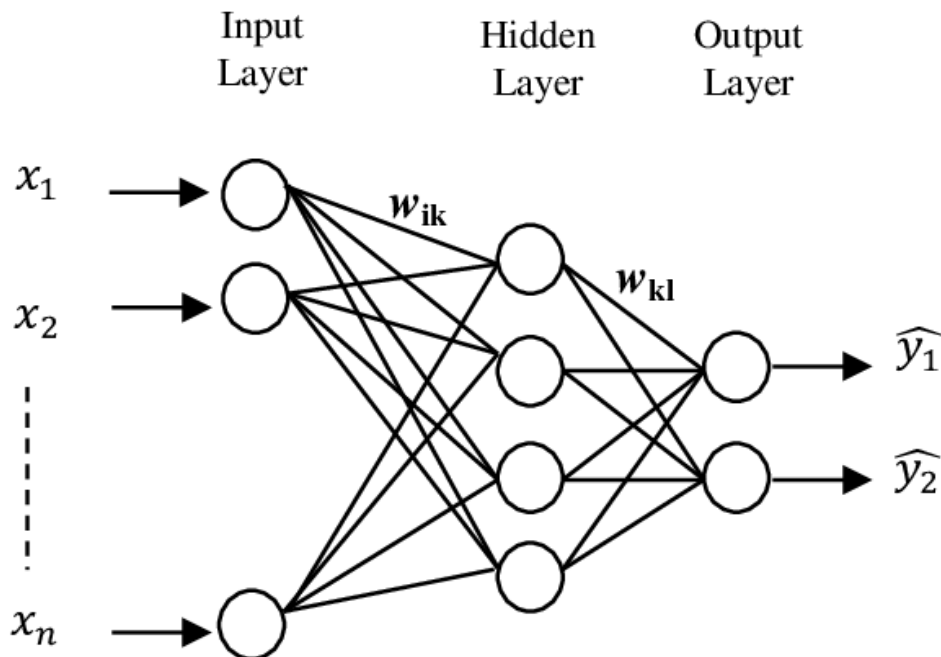


Figure 3 - Multi Layer Perceptron

I used **Adam optimizer** that has a 10^{-3} learning rate and tracked the AUC metric. I ran 12 epochs with 256 batches and 10% validation set that I took from combined data. I used it in a way to address the non-linearities in the features, just to see how it performs. This gave me the drive to build a complex hybrid model which I will discuss in detail in the next section.

3.6 LeagueEWS architecture:

I designed a model from scratch and named it as **LeagueEWS**. It combines Temporal convolutions, recurrent layers, multi-head attention and a shared representation that has three output heads for Baron, Dragon and teamfights.

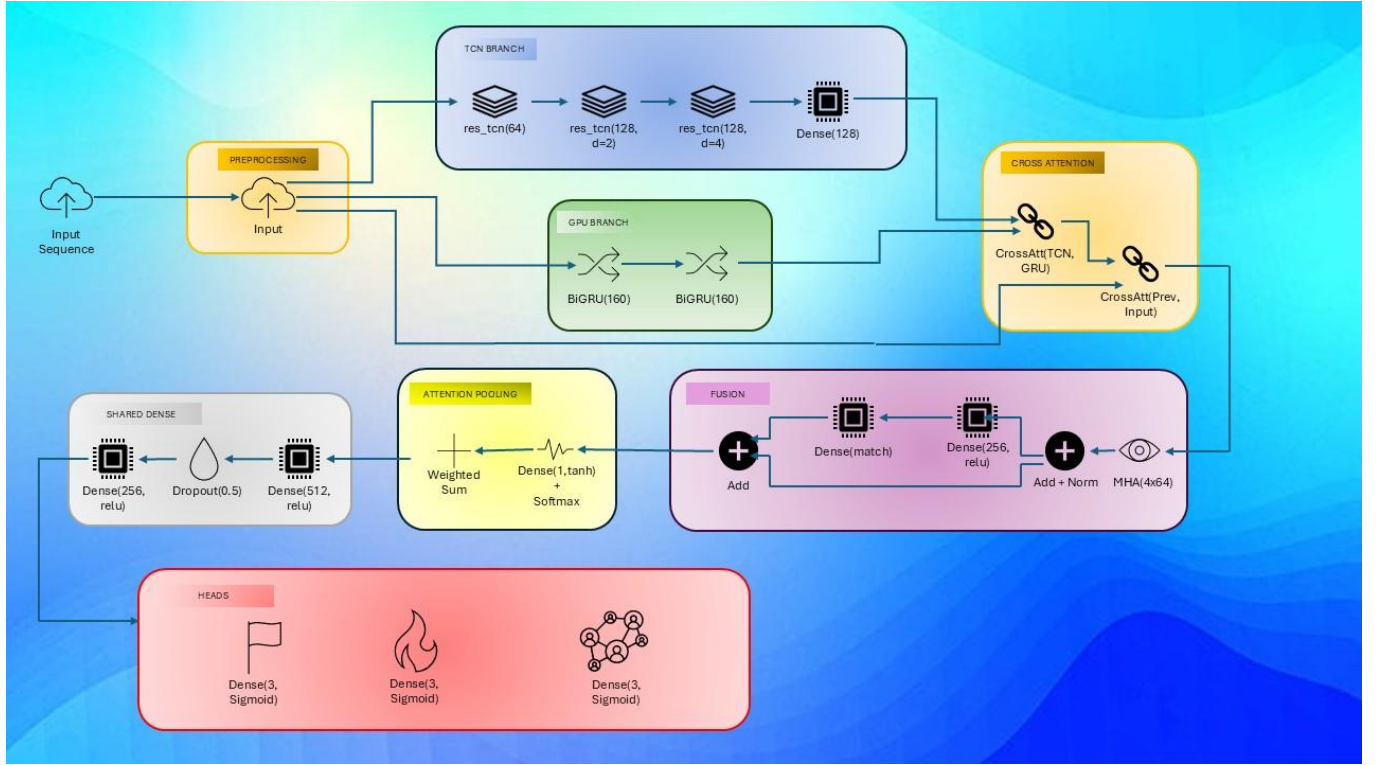


Figure 4 - LeagueEWS architecture

The first mix of the architecture I used is a temporal convolutional network with residual connections. It has several 1D convolutional blocks with increasing dilation factors (**1, 2, and 4**) which allow the network to inspect different timeframes. I applied a convolution, batch normalization and dropout on each residual block, which is followed by a second convolution and batch normalization again. Then, I used a squeeze-excitation mechanism to compute a global average over time, then pass it through two dense layers with ReLU (ϕ) and sigmoid (σ) activations and used the resulting vector to rescale the features. I made a 1x1 convolution to align the input and output channel dimensions if it does not match. And I completed it with a ReLU activation.

$$s = \frac{1}{T} \sum_{t=1}^T H_t, \quad u = \phi(W_1 s + b_1), \quad v = \sigma(W_2 u + b_2)$$

I made a two stacked Bidirectional GRU layers with 160 units to run the inputs in parallel in each direction. It's well suited to capture many sequential patterns that occurs, also the prolonged changes that's tough to express with only local convolutions. I added the cross attention mechanism to make the TCN's path and GRU's path parallel to give outputs. The **Multi-Head Attention Layer** (MHA) will take the TCN output as queries and GRU output as keys and values, that gives a combined representation. Then it will be taken back to TCN activations and gets normalized. I added a second cross attention block to make the combined sequence go back to raw input, then getting normalized again.

$$G = \text{BiGRU}_2 (\text{BiGRU}_1 (X))$$

$$A_1 = \text{LayerNorm} (\text{TCN}(X) + \text{MHA}(\text{TCN}(X), G))$$

I made a self-attention layer for those combined sequences. Then a simple feed-forward network will wrap the result in a residual connection. I included a logic to condense the 40 timesteps into a single vector, where the model predicts attention score for each frame, then normalizes that scores with a **SoftMax** (w_t) overtime, computing a **weighted sum of hidden states** ($H = A_3 \in \mathbb{R}^{T \times d}$). I don't want the

model to depend on the final frame, so I added an Attention pooling step that allow the network to focus on most informative frames in the next few seconds.

$$\begin{aligned} A_2 &= \text{LayerNorm} (A_1 + \text{MHA} (A_1, X)) \\ A_3 &= \text{LayerNorm} (A_2 + \text{MHA} (A_2, A_2)) \\ a_t &= w^T H_t \text{ for } t = 1..., T \end{aligned}$$

For the softmax activation,

$$w_t = \frac{\exp(a_t)}{\sum_{s=1}^T \exp(a_s)}$$

Entire pooled representation

$$h^* = \sum_{t=1}^T w_t H_t$$

I put this entire representation through a dense layer that has 512 ReLU units, a L2 regularization, followed by dropout, and then through a second dense layer with 256 ReLU units. I made three heads, a Baron head, a Dragon head, and a Teamfight head, each having a dense layer with three sigmoid outputs for each horizon ($h \in \{10, 20, 30\}$). The model computes **logit** $z^{(e,h)}$ and probability,

$$\hat{p}^{(e,h)} = \sigma(z^{(e,h)})$$

I used binary cross-entropy (L_e) as loss function where, λ_{baron} is 1.0, λ_{dragon} is 2.0, $\lambda_{\text{teamfight}}$ is 2.5.

$$L_{\text{LeagueEWS}} = \lambda_{\text{baron}} L_{\text{baron}} + \lambda_{\text{dragon}} L_{\text{dragon}} + \lambda_{\text{teamfight}} L_{\text{teamfight}}$$

3.7 Hyperparameter tuning and training:

I added few hyperparameters in my LeagueEWS model and did a random search with **Keras Tuner**. It gave me numbers for TCN filters, GRU units, dropout rates and initial learning rates. My goal was to maximise the mean validation AUC of all three events.

$$\text{val_mean_auc} = \frac{1}{3} (AUC_{\text{baron}} + AUC_{\text{dragon}} + AUC_{\text{teamfight}})$$

After running for 5 trails, I found mix of the suitable numbers,

Table 3 - Tuning results

trial_id	val_mean_auc	tcn_filters	gru_units	shared_dropout	learning_rate
2	0.875639	80	160	0.5	0.00010
0	0.857301	48	160	0.4	0.00007
3	0.855176	80	128	0.5	0.00005
4	0.852442	48	128	0.3	0.00005
1	0.851459	64	96	0.5	0.00005

I used these settings for model training for **30 epochs** using AdamW optimizer. Then, the learning rate followed by cosine decay schedule with a floor $\alpha = 0.2$. If I denote s as the training step and S as the whole number of decay steps, the learning rate at step s would be,

$$\eta(s) = \eta_0 \left[\alpha + (1 - \alpha) \frac{1 + \cos\left(\pi \frac{s}{S}\right)}{2} \right]$$

where $\eta_0 = 10^{-4}$ and $\alpha = 0.2$.

Looking at the epoch curves, there's a sharp jump in AUC and drop in loss during the first five to eight epochs, then it improved as learning rate decays. The gap between train and validation is telling me the regularization and that huge dropout does its job. After twenty epochs, the curve starts to flatten, reaching a plateau. So, I just kept it under thirty epochs.

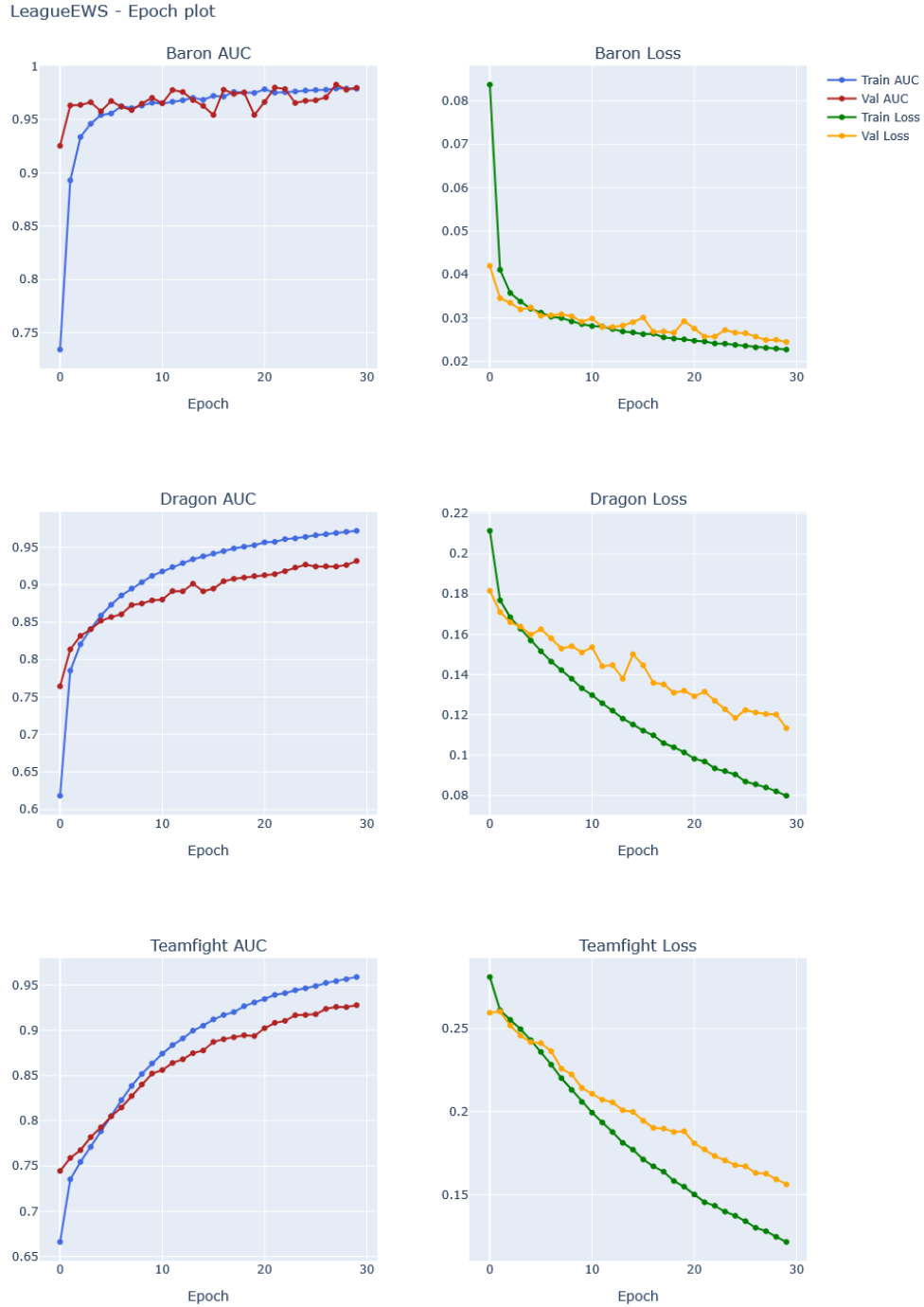


Figure 5 - Epoch plot for LeagueEWS

3.8 Evaluation metrics:

To check the performance of my models I used three metrics: ROC AUC, Brier score and precision at a fixed probability threshold. I used brier score to measure the mean squared error between the predicted probabilities and true labels. For an even with N samples it is,

$$Brier = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i)^2$$

Lower the values, better the probabilistic accuracy. Next, I used Precision to look at the quality of positive predictions for a chosen threshold (T = 0.5).

$$Precision = \frac{TP}{TP + FP}$$

Then, I used ROC AUC to check how the model orders the positive examples above negatives for all available thresholds. If $P = \{i: y_i = 1\}$ and $N = \{j: y_j = 0\}$, then

$$AUC = \frac{1}{|P| |N|} \sum_{i \in P} \sum_{j \in N} 1[\hat{p}_i > \hat{p}_j]$$

where $1[.]$ is indicator function. Usually, people compute AUC via **trapezoidal approximation** to the ROC curve, but this expression captures the **underlying area**. I computed these metrics for each event on test set.

4. RESULTS:

4.1 Overall predictive performance:

As I expected, my hybrid model LeagueEWS outperforms both my reference models in all the metrics.

Table 4 – Evaluation metrics of all models

Model	Event	AUC	Precision	Brier
Random Forest	Baron	0.97	0.05	0.09
	Dragon	0.92	0.20	0.16
	Teamfight	0.70	0.14	0.22
Simple MLP	Baron	0.97	0.35	0.008
	Dragon	0.73	0.20	0.16
	Teamfight	0.74	0.51	0.07
LeagueEWS	Baron	0.98	0.70	0.006
	Dragon	0.95	0.72	0.03
	Teamfight	0.92	0.74	0.04

When the Random Forest achieved strong AUC for baron and dragon, it confirmed that the static game statistic contains a lot of signals. But the imbalance affected the precision and brier score a lot. So, to improve this, I used a simple multi-layer perceptron to boost the precision for the events and surprisingly it did a fair job.

But the hybrid model I made improved all three metrics, where baron is strong with 0.98 AUC with a very low brier score and precision around 0.7. Dragon and teamfight with its chaotic event nature, the AUC is >0.90 and a precision of >0.7, showing that my model LeagueEWS can perform better while the game is extremely **short spiked** in these events.

4.2 ROC curves:

I plotted these curves for all three models to see how each model **trades off sensitivity** and false-positive rate. League AWS pushes the curve the closest to the upper left corner, showing it sustain a high true-positive rate even at very low false-positive rates, telling me that it will be both selective and accurate.

ROC curves – Random Forest

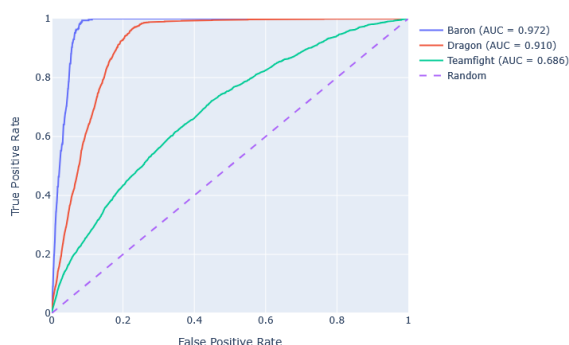


Figure 6 – RF ROC curve

ROC curve - Simple MLP

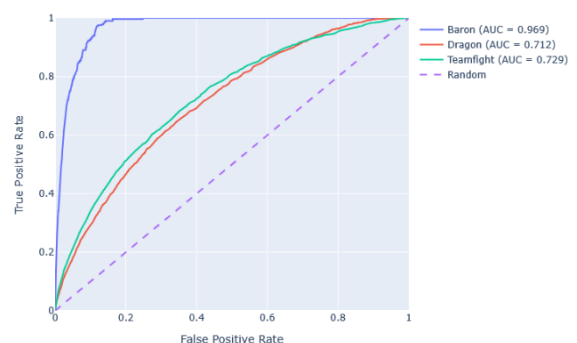


Figure 7 – Simple MLP ROC curve

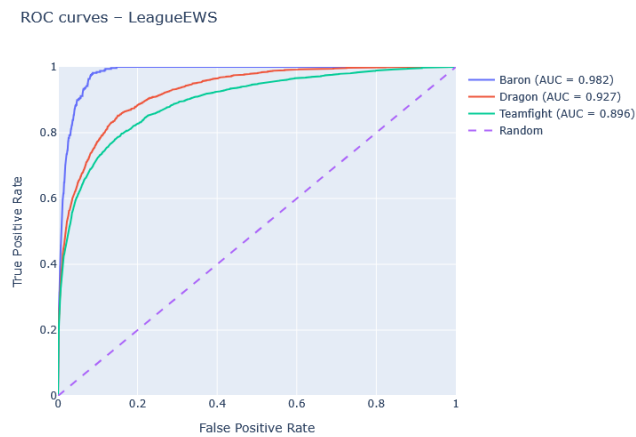


Figure 8 - LeagueEWS ROC curve

What I see as a main gain is the FPR ranging between **0.1 and 0.3**, where LeagueEWS pick up more true positives while others flatten out. In League game, the teamfights are almost unpredictable, but initially while setting a goal for this project, I wanted to give a shot on predicting them to some extent. The reference models struggled a lot and failed to match my vision. But LeagueEWS produced the best curve I can think of, with an AUC close to 0.90 – 0.92 and a much a steeper rise near origin, showing for an alarm system, it is discriminating better to keep false positives low.

4.3 Confusion matrices:

The dataset I took is 10-second sequential snapshot, and a match average time is around **35 minutes**, so even when we build sequences in the data preprocessing, there will be a lot of neutral states, where the events won't happen. Even though, I plotted the confusion matrix to give a justification for the precision I'm trying to achieve. And I'm shooting for an alarm system, so if I give out a lot of false alarms, it will not be appropriate tool for the players to learn. If you look at the Random Forest, it leaves out a lot of positive cases as false ones. The same happened in the MLP, where it did improve but not as a suitable model for an alarm system. But LeagueEWS manages to increase the true positives while keeping the false positives down only at the near windows.

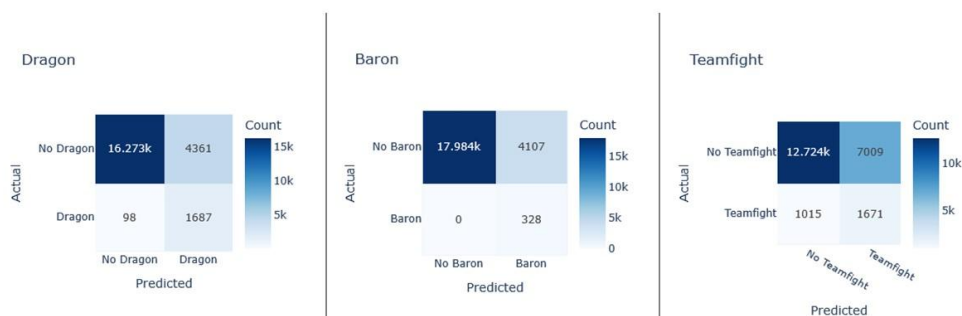


Figure 9 - RF confusion matrix



Figure 10 - Simple MLP confusion matrix

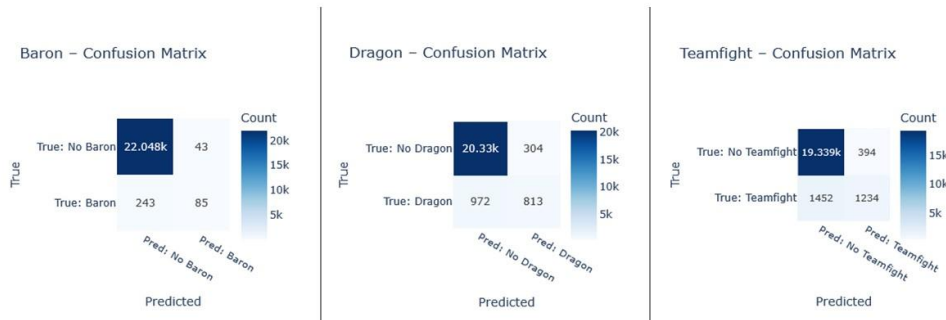


Figure 11 - LeagueEWS confusion matrix

4.4 Risk plots:

After confirming how my model performs through the metrics, I wanted to see how the signals look like over the entire match. So, I made risk plots for individual games consists of all three events. For the reference models, the performance was bad for dragon and teamfight, but it did a good job for baron event. In the plots, the **light red band** is the actual window of the events in historical data. The **yellow star** denotes that it's a true alarm and **red star** denotes that it's a false alarm.

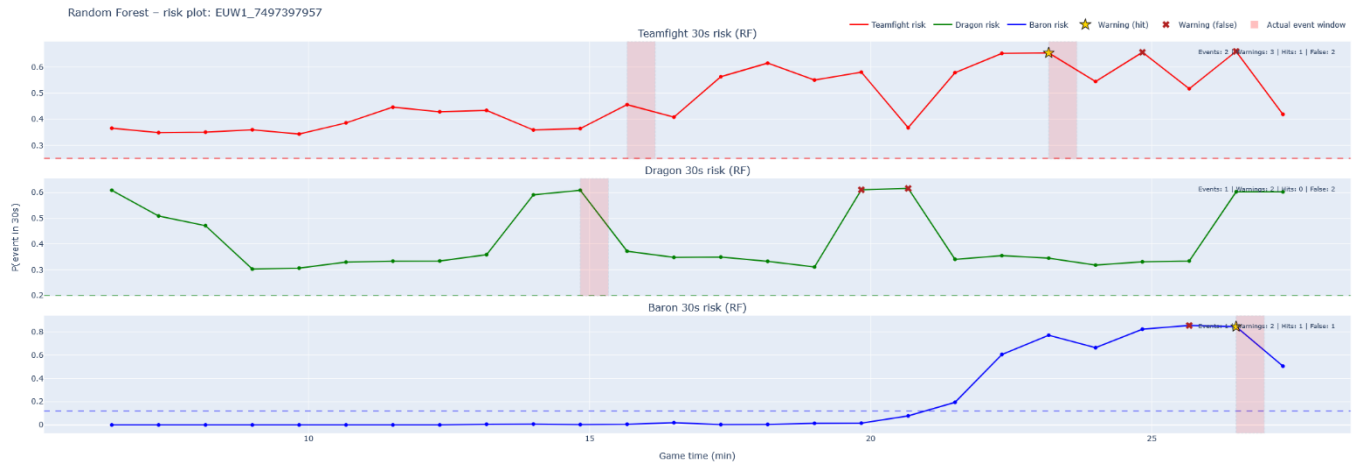


Figure 12 - RF risk plot

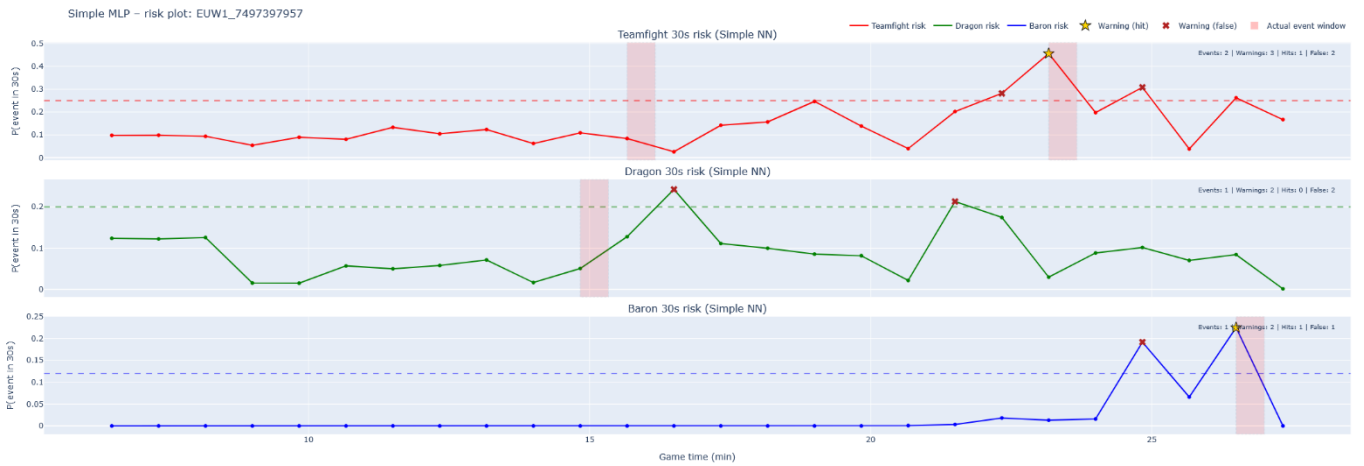


Figure 13 - Simple MLP risk plot

But LeagueEWS, produced some accurate alarms, and some near window alarms. Comparing these plots, it suggests that this model converts its strong AUC and calibration into practically useful behaviour. This model is suitable for an actual alarm system.

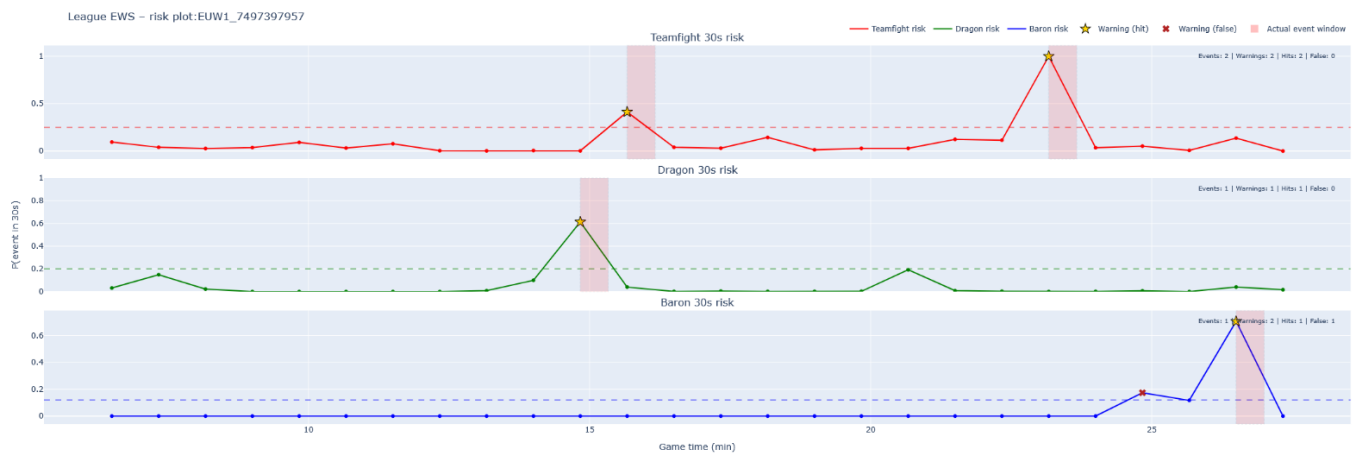


Figure 14 - LeagueEWS risk plot

5. ETHICAL CONSIDERATIONS:

I made this project focussing on a video game, so it raises a few ethical questions that is complicated. The major concern is fairplay, where a real-time version of early warning system could be considered as cheating by most game publishers. So, the development of this system is only limited to offline analysis, coaching or research environments, rather than for live gameplay.

And the data privacy is another aspect. The dataset I tailored only consists of in-game statistics which does not include real-world personal identifiers. Since I extracted the raw data through [official Riot API](#), the privacy regulations are followed without any error. And the input data are only at the level of teams and matches and there's no attempt to rank individual players with model predictions.

And the decision support systems that could be built on this model also influence behaviour of the players. So, when it's used for coaching, it should only be presented as probabilistic hints to look the triggers of the events, where the players could learn without getting influenced. And coaches are to be given an awareness that the model can be wrong in extremely chaotic situations, where their own judgement and experience speak more there.

And the techniques I used here such as sequence modelling, attention mechanism and early warning logic are slightly similar to the ones used in some domains like healthcare or finance. But my project is in the entertainment, and it does indirectly illustrate both the promise and risk of such methods. So, if there is going to be transfer of ideas, it should require strict focus on validation, interpretability, robustness and accountability than something circles around a game.

6. DISCUSSION:

6.1 Interpreting the gains:

When I compared the reference models and LeagueEWS, it gives a narrative. By simply looking at a single frame, there's a lot of reasonable information carried. Random Forest achieved strong AUC values, especially for Baron and Dragon, which tells me that many factors of these events can be seen in current state alone. But it struggles a lot when it comes to calibration and shows poor performance when comes to the ability to decide, because of the class imbalance.

While being dissatisfied, I used a small non-linear model in the form of simple MLP that majorly fixed the calibration and improved precision. But again, it didn't match what I had in my vision. So, I experimented n number of times to build a custom model mixing unique ingredients to LeagueEWS, namely temporal context and multi-task learning. The network inspects the previous 40 frames to look at the relevant ones, which makes a huge difference when it comes to events like teamfights where the buildup is significant as much as final deployment. The attention mechanism is effective since it relieves the model from compressing the entire history into last hidden state. The events I tried to predict rely on similar factors like rotations, objective control and power spikes. So, learning these factors once and specialising only at the output heads gives good generalisation than to train separate models.

Out of these events, predicting Baron is easiest. The baron attempts are rare that happens in the late game and a bit easy to pinpoint it. The dragon attempts happen in cycle and its importance changes throughout the match. Teamfights are chaotic that's illusional with the positional factors. Even with these constrictions, LeagueEWS achieves an AUC above 0.9 for teamfights, while the reference models only got closer to 0.7. So, the custom model learned to interpret more subtle pre-fight patterns that's not obvious in a single snapshot.

6.2 Improvement of models:

- The Random Forest could achieve calibrated probabilities on rare Baron and Dragon events by doing a grid or random search over depth, number of trees, or trying gradient boosting like XGBoost.
- I made a simple MLP which only see the final time step, so I could replace it with a small one-dimensional CNN or GRU for the whole sequence while keeping it lightweight and a bit aware.
- I did use jittering to address the class imbalance issue, but I could test focal loss or do hard negative mining where the model at near windows instead of random negatives.
- The LeagueEWS model could be simplified by first training the full TCN + BiGRU + attention model then trains a small network to just imitate its probabilities. This would make it lighter and faster.

6.3 Limitations:

The results are good, but it does have its limitations. The dataset I tailored is tied to a particular patch, region and player population. So, the model behaviour might change in other regions or with different skill levels, because the game shifts in high tier ranked matches differ a lot than the low tier ranked matches. The intensity of teamfight might shift when there is counter champion picked by the enemy team, where it could change the difficulty of the task and the numerical results.

The sliding window is shorter where it focuses on immediate moments during the objective trigger. But, in longer windows, it will cover macro rotations and lane states leading up to them. Also, the model sees features from the game state, but not the champion composition, win conditions or timers. So, with more structured representation, the future models will be more robust and interpretable. In overall, instead of having definitive numbers, the insights should be seen as evidence that temporal, multi-task sequence models can deliver strong early warning performance in this setting.

7. CONCLUSION:

I started this project with an idea to build an early warning system for League of Legends to predict in-game events like Baron, Dragon and Teamfight, which can be deployable now. After designing the structured dataset of match timelines, I cleaned and reshaped the data into sequences of 40 timesteps, and added temporal aware features like combat, objective history and team clustering. I used jittering to reduce the class imbalance issue so the model can differentiate events without going for unrealistic states.

For modelling, I compared a Random Forest and a simple feed-forward network that only saw the last timestep against a custom LeagueEWS model that I built from TCN blocks, bidirectional GRUs and cross attention. The reference models did an average performance where the precision fell short badly that's not suitable for an alarm system. But LeagueEWS achieved higher AUC, precision and lower brier scores and produced better risk plots. The work does have limitations which is bearable, so the system is promising and usable as a learning tool or an early-production grade which needs further development.

References

- Bahrololloomi, F., Klonowski, F., Sauer, S., Horst, R., & Dörner, R. (2023). E-Sports Player Performance Metrics for Predicting the Outcome of League of Legends Matches Considering Player Roles. *SN Computer Science*, 4, 238. Retrieved from <https://doi.org/10.1007/s42979-022-01660-6>
- Chang, Y. (.), Oh, D., & Madden, C. (2008). Jitter modeling in statistical link simulation. *IEEE International Symposium on Electromagnetic Compatibility* (pp. 814–817). Detroit, MI: IEEE. Retrieved 12 15, 2025, from https://www.researchgate.net/publication/224340733_Jitter_modeling_in_statistical_link_simulation
- Cong, S., Wu, H., Xiong, S., Wu, Y., & Zuo, L. (2024). Predicting the Outcome of MOBA Game: a Case Study on Pregame Prediction for League of Legends. *2024 8th Asian Conference on Artificial Intelligence Technology (ACAIT)* (pp. 1366-1372). IEEE. Retrieved from <https://doi.org/10.1109/ACAIT63902.2024.11021901>
- Hitar-García, J. A., Morán-Fernández, L., & Bolón-Canedo, V. (2023). Machine learning methods for predicting League of Legends game outcome. *IEEE Transactions on Games*, 15(2), 171–181. Retrieved from <https://doi.org/10.1109/TG.2022.3153086>
- Hojaji, F., McIlroy, R. E., Dupuy, A., Pedroni, G., Toth, A. J., & Campbell, M. J. (2025). Deep learning techniques for identifying KPIs in League of Legends: Win prediction, map navigation, and vision control. *Computers in Human Behavior Reports*, 19, 100718. Retrieved from <https://doi.org/10.1016/j.chbr.2025.100718>
- Junior, J. B., & Campelo, C. E. (2023). League of Legends: Real-Time Result Prediction. *IEEE Conference Proceedings*. IEEE. Retrieved from <https://arxiv.org/abs/2309.02449>
- Liu, C., Yin, Y., Sun, Y., & Ersoy, O. (2022). Multi-scale ResNet and BiGRU automatic sleep staging based on attention mechanism. *PLOS ONE*, 17(6), e0269500. Retrieved 12 15, 2025, from <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0269500>
- Louppe, G. (2014). *Understanding Random Forests: From Theory to Practice*. arXiv. Retrieved 12 15, 2025, from <https://arxiv.org/abs/1407.7502>
- (n.d.). *Riot Developer Portal*. Retrieved from <https://developer.riotgames.com/>
- Tolstikhin, I., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., . . . Dosovitskiy, A. (2021). *MLP-Mixer: An all-MLP Architecture for Vision*. arXiv. Retrieved 12 15, 2025, from <https://arxiv.org/abs/2105.01601>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., . . . Polosukhin, I. (2017). *Temporal Convolutional Networks for Action Segmentation and Detection*. arXiv. Retrieved 12 15, 2025, from <https://arxiv.org/abs/1611.05267>
- Wang, Y., Wu, H., Dong, J., Liu, Y., Wang, C., Long, M., & Wang, J. (2024). *Deep Time Series Models: A Comprehensive Survey and Benchmark*. arXiv. Retrieved December 15, 2025, from <https://arxiv.org/abs/2407.13278>

Appendix:

Dataset Link



GitHub Link



Code – EDA

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv("/content/drive/MyDrive/final_dataset.csv")
df.sample(5)
```

```
df.info()
```

```
df.describe().T
```

```
df.shape
```

```
dup_full = df.duplicated().sum()
dup_keys = df.duplicated(subset=["match_id", "t"]).sum()
```

```
print("Duplicate full rows:", dup_full)
print("Duplicate (match_id, t) rows:", dup_keys)
```

```
missing_counts = df.isnull().sum()
missing_percent = (df.isnull().mean() * 100).round(4)
```

```
print("Missing value counts:\n", missing_counts)
print("\nMissing value %:\n", missing_percent)
```

```
cols = df.columns.tolist()
```

```
gold_cols = ["gold_blue", "gold_red", "gold_diff"]
```

```
xp_cols = ["xp_blue", "xp_red", "xp_diff"]
```

```
cs_cols = ["cs_lane_t1", "cs_lane_t2", "cs_jungle_t1", "cs_jungle_t2", "cs_total_t1", "cs_total_t2",
"cs_total_diff"]
```

```
level_cols = ["avg_level_t1", "avg_level_t2", "level_diff"]
```

```
status_cols = ["alive_t1", "alive_t2", "alive_diff", "low_hp_t1", "low_hp_t2", "group_t1", "group_t2",
"cluster_density", "distance_between_teams", "spread_t1", "spread_t2"]
```



```

objective_position_cols = ["champs_near_baron_t1","champs_near_baron_t2",
"champs_near_dragon_t1","champs_near_dragon_t2"]

combat_30_cols = ["kills_30_t1","kills_30_t2", "deaths_30_t1","deaths_30_t2"]

combat_60_cols = ["kills_60_t1","kills_60_t2", "deaths_60_t1","deaths_60_t2",
"wards_placed_60_t1","wards_placed_60_t2", "wards_killed_60_t1","wards_killed_60_t2",
"items_purchased_60_t1","items_purchased_60_t2",]

objective_taken_cols = ["dragons_t1","dragons_t2", "barons_t1","barons_t2", "heralds_t1","her-
alds_t2", "towers_t1","towers_t2", "inhibs_t1","inhibs_t2"]

timer_cols = ["time_since_last_baron", "time_since_last_dragon", "time_since_last_teamfight",
"teamfight_recent_20"]

label_cols = ["baron", "dragon", "teamfight"]

combat_raw_cols = ["kills_blue","kills_red", "deaths_blue","deaths_red", "assists_blue","as-
sists_red"]

damage_cols = ["totdmgdealt_blue","totdmgdealt_red", "totdmgtochamp_blue","totdmgto-
champ_red", "toheal_blue","toheal_red"]

vision_cols = ["wardsplaced_blue","wardsplaced_red", "wardskilled_blue","wardskilled_red"]


print("\nECONOMY:", gold_cols + xp_cols)
print("\nCS:", cs_cols)
print("\nLEVELS:", level_cols)
print("\nTEAM STATE:", status_cols)
print("\nPOSITION PRESSURE:", objective_position_cols)
print("\nCOMBAT (30s):", combat_30_cols)
print("\nCOMBAT (60s):", combat_60_cols)
print("\nOBJECTIVES TAKEN:", objective_taken_cols)
print("\nTIMERS:", timer_cols)
print("\nLABELS:", label_cols)
print("\nCOMBAT RAW:", combat_raw_cols)
print("\nDAMAGE:", damage_cols)
print("\nVISION:", vision_cols)

cols_nonneg = (
    gold_cols + xp_cols + cs_cols + level_cols + status_cols +
    objective_position_cols + combat_30_cols + combat_60_cols +
    objective_taken_cols + timer_cols + combat_raw_cols +
    damage_cols + vision_cols
)

```

```

for col in cols_nonneg:
    if col in df.columns:
        neg_count = (df[col] < 0).sum()
        if neg_count > 0:
            print(f"{col}: {neg_count} negative values found")
        else:
            print(f"{col}: column missing in dataset")
for col in vision_cols:
    if col in df.columns:
        fractional = (df[col] % 1 != 0).sum()
        if fractional > 0:
            print(f"{col}: {fractional} rows have fractional values")
        else:
            print(f"{col}: column missing")
bad = 0

for mid, grp in df.groupby("match_id"):
    diffs = grp["t"].diff().dropna()
    if len(diffs) > 0:
        if not (diffs == diffs.iloc[0]).all():
            bad += 1

print("Matches with inconsistent timestamp intervals:", bad)
counter_cols = combat_raw_cols

non_mono = 0

for col in counter_cols:
    for mid, grp in df.groupby("match_id"):
        d = grp[col].diff().fillna(0)
        if (d < 0).any():
            print(f"{col} decreases in match {mid}")

```

```

    non_mono += 1
    break

print("Columns with monotonicity issues:", non_mono)

# remove duplicates
df = df.drop_duplicates()
df = df.drop_duplicates(subset=["match_id", "t"])

# sort properly
df = df.sort_values(["match_id", "t"]).reset_index(drop=True)
nonneg_cols = (
    gold_cols + xp_cols +
    cs_cols + level_cols +
    status_cols + objective_position_cols +
    combat_30_cols + combat_60_cols +
    objective_taken_cols + timer_cols +
    combat_raw_cols + damage_cols + vision_cols
)

# clip negatives to zero
for c in nonneg_cols:
    if c in df.columns:
        df[c] = df[c].clip(lower=0)

for c in vision_cols:
    if c in df.columns:
        df[c] = df[c].round().astype(int)

# recompute gold and xp differences
if "gold_blue" in df.columns and "gold_red" in df.columns:
    df["gold_diff"] = df["gold_blue"] - df["gold_red"]

if "xp_blue" in df.columns and "xp_red" in df.columns:
    df["xp_diff"] = df["xp_blue"] - df["xp_red"]

```

```
weird_matches = 0
```

```
for mid, grp in df.groupby("match_id"):
```

```
    diffs = grp["t"].diff().dropna()
```

```
    if not (diffs == diffs.iloc[0]).all():
```

```
        weird_matches += 1
```

```
print("Matches with inconsistent t-spacing:", weird_matches)
```

```
kill_cols = [
```

```
    "kills_blue", "kills_red",
```

```
    "kills_30_t1", "kills_30_t2",
```

```
    "kills_60_t1", "kills_60_t2"
```

```
]
```

```
kill_issues = 0
```

```
for c in kill_cols:
```

```
    if c not in df.columns:
```

```
        continue
```

```
    for mid, grp in df.groupby("match_id"):
```

```
        if (grp[c].diff().fillna(0) < 0).any():
```

```
            kill_issues += 1
```

```
print("Matches where kill count decreases:", kill_issues)
```

```
print("Final dataset shape:", df.shape)
```

```
print("\nRemaining missing values:", df.isnull().sum().sum())
```

```
from plotly.subplots import make_subplots
```

```
import plotly.graph_objects as go
```

```
import plotly.express as px
```

```
import pandas as pd
```

```
df["baron"] = df[["y_baron_10", "y_baron_20", "y_baron_30"]].mean(axis=1)
```

```
df["dragon"] = df[["y_dragon_10", "y_dragon_20", "y_dragon_30"]].mean(axis=1)
```

```
df["teamfight"] = df[["y_teamfight_10", "y_teamfight_20", "y_teamfight_30"]].mean(axis=1)
```

```

df["t_min"] = df["t"] // 60

df_time = df.groupby("t_min")[["baron", "dragon", "teamfight"]].mean().reset_index()

fig = make_subplots(rows=3, cols=1, shared_xaxes=True, subplot_titles=["Baron Probability",
"Dragon Probability", "Teamfight Probability"])

events = ["baron", "dragon", "teamfight"]
colors = ["blue", "green", "red"]

for i,(ev,c) in enumerate(zip(events,colors), start=1):

    fig.add_trace(go.Scatter(x=df_time["t_min"], y=df_time[ev], mode="lines", line=dict(color=c),
                            name=ev.capitalize()), row=i, col=1)

fig.update_layout(height=900, title="Major events over time throughout the match", tem-
plate="plotly_white")

fig.update_xaxes(title="Time (min)", row=3)

fig.update_yaxes(title="Probability")

fig.show()

roll = df.groupby("t_min")[["gold_diff", "xp_diff"]].mean().rolling(5).mean().reset_index()

fig = go.Figure()

fig.add_trace(go.Scatter(x=roll["t_min"], y=roll["gold_diff"], name="Gold Diff", mode="lines",
line=dict(color="gold"))))

fig.add_trace(go.Scatter(x=roll["t_min"], y=roll["xp_diff"], name="XP Diff", mode="lines",
line=dict(color="blue"))))

fig.update_layout(title="Gold and XP Advantage Trajectory Over Match Duration", xaxis_title="Time
(min)",

                  yaxis_title="Difference", template="plotly_white", height=400)

fig.show()

df["vision_diff"] = df["wardsplaced_blue"] - df["wardsplaced_red"]

vision_time = df.groupby("t_min")["vision_diff"].mean().reset_index()

fig = px.line(vision_time, x="t_min", y="vision_diff", title="How Vision Control Shifts Over Time",

              labels={"t_min": "Time (min)", "vision_diff": "Vision Difference (Blue - Red)"}, tem-
plate="plotly_white")

fig.add_annotation(x=0.02, y=1.05, xref="paper", yref="paper", text="Positive = Blue has more vi-
sion", showarrow=False, font=dict(color="blue", size=12))

```

```

fig.add_annotation(x=0.02, y=1.00, xref="paper", yref="paper", text="Negative = Red has more vi-
sion", showarrow=False, font=dict(color="red", size=12))

fig.show()

def phase(t):
    if t < 600: return "Early"
    elif t < 1500: return "Mid"
    return "Late"

df["phase"] = df["t"].apply(phase)

for event in ["baron", "dragon", "teamfight"]:
    phase_counts = df[df[event] == 1]["phase"].value_counts().reset_index()
    phase_counts.columns = ["Phase", "Count"]
    fig = px.pie(phase_counts, names="Phase", values="Count", title=f"Game Phases When {event.cap-
italize()} Happens", hole=0.4)
    fig.show()

df["cluster_bin"] = pd.qcut(df["cluster_density"], q=5, labels=["Very Low", "Low", "Medium", "High",
"Very High"])

df_cluster = (df.groupby("cluster_bin")[["baron", "dragon", "teamfight"]].mean().reset_index())

bin_map = {
    "Very Low": "Spread Out",
    "Low": "Loose Grouping",
    "Medium": "Approaching",
    "High": "Heavy Grouping",
    "Very High": "All-In Cluster"
}

df_cluster["cluster_bin"] = df_cluster["cluster_bin"].map(bin_map)

df_melt = df_cluster.melt(id_vars="cluster_bin", value_vars=["baron", "dragon", "teamfight"],
var_name="event", value_name="probability")

color_map = {"baron": "blue", "dragon": "green", "teamfight": "red"}

fig = px.bar(df_melt, x="cluster_bin", y="probability", color="event", barmode="group",
text=df_melt["probability"].round(3), color_discrete_map=color_map,

```

```
title="Impact of Team Clustering on Major Event Probability",labels={"cluster_bin": "Team  
Clustering Level", "probability": "Probability (next 30s)", "event": "Event Type"})
```

```
fig.update_layout(template="plotly_white", height=500, width=950, title_font_size=20, legend_title_text="Event", xaxis_tickangle=-20,  
  
xaxis=dict(title="Clustering Level (Spread to Fully Grouped)"), yaxis=dict(title="Event Probability"))  
  
fig.update_traces(textposition="outside")  
  
fig.show()
```

Code – Model Building

```
!pip install keras-tuner --quiet  
import numpy as np  
import pandas as pd  
import tensorflow as tf  
from tensorflow.keras import layers, models, mixed_precision  
from sklearn.preprocessing import StandardScaler  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import (precision_score, recall_score, log_loss, roc_curve, roc_auc_score,  
brier_score_loss, precision_recall_curve, confusion_matrix,)  
import plotly.express as px  
import plotly.graph_objects as go  
from plotly.subplots import make_subplots  
from sklearn.ensemble import RandomForestClassifier  
from tensorflow.keras import regularizers  
from tensorflow.keras.metrics import AUC  
mixed_precision.set_global_policy("mixed_float16")  
df = pd.read_csv("/content/drive/MyDrive/final_dataset.csv")  
df = df.sort_values(["match_id", "t"])  
df = df.fillna(0)  
df.sample(5)  
target_columns = ["y_baron_10", "y_baron_20", "y_baron_30", "y_dragon_10", "y_dragon_20",  
"y_dragon_30", "y_teamfight_10", "y_teamfight_20", "y_teamfight_30"]  
  
exclude_columns = ["match_id", "t"]  
for col in target_columns:  
    exclude_columns.append(col)  
  
feature_columns = []  
for col in df.columns:  
    if col not in exclude_columns:  
        feature_columns.append(col)  
  
for col in feature_columns:  
    new_col_name = col + "_diff"  
    tmp = df.groupby("match_id")[col].diff()  
    tmp = tmp.fillna(0)  
    df[new_col_name] = tmp
```

```

feature_columns = []
for col in df.columns:
    if col not in exclude_columns:
        feature_columns.append(col)

print("Num features:", len(feature_columns))
print("Num targets:", len(target_columns))
scaler = StandardScaler()
df[feature_columns] = scaler.fit_transform(df[feature_columns])
SEQ_LEN = 40
STEP = 5

def make_seq(df, feature_columns, target_columns, seq_len=40, step=5):
    xs = [] # all sequences from all matches
    ys = [] # all targets from all matches

    for match_id, g in df.groupby("match_id"):
        g = g.sort_values("t")
        xf = g[feature_columns].values.astype(np.float32)
        yt = g[target_columns].values.astype(np.float32)

        if len(xf) < seq_len:
            continue

        seq_list = [] # sequences just for this match
        targ_list = [] # targets just for this match

        for start_idx in range(0, len(xf) - seq_len + 1, step):
            end_idx = start_idx + seq_len

            x_window = xf[start_idx:end_idx]
            y_window = yt[end_idx - 1]

            seq_list.append(x_window)
            targ_list.append(y_window)

        seq_arr = np.array(seq_list, dtype=np.float32)
        targ_arr = np.array(targ_list, dtype=np.float32)

        xs.append(seq_arr)
        ys.append(targ_arr)

    X_all = np.concatenate(xs, axis=0)
    Y_all = np.concatenate(ys, axis=0)

    return X_all, Y_all

def jitter_seq(x, y, shift=2, repeat=2):
    n, t, f = x.shape

    xs = [x]

```



```

ys = [y]

for _ in range(repeat):
    shifts = np.random.randint(-shift, shift + 1, size=n)
    pad_x = np.pad(x, pad_width=((0, 0), (shift, shift), (0, 0)), mode="reflect")
    new_x = np.zeros_like(x)

    for i in range(n):
        s = shifts[i]
        start_idx = s + shift
        end_idx = start_idx + t
        new_x[i] = pad_x[i, start_idx:end_idx, :]

    xs.append(new_x)
    ys.append(y)

X_out = np.concatenate(xs, axis=0)
Y_out = np.concatenate(ys, axis=0)

return X_out, Y_out

X_seq, y_seq = make_seq(df, feature_columns, target_columns, seq_len=SEQ_LEN, step=STEP)
print("Raw sequences:", X_seq.shape, y_seq.shape)

X_aug, y_aug = jitter_seq(X_seq, y_seq, shift=2, repeat=2)
print("Augmented sequences:", X_aug.shape, y_aug.shape)
X_trainval, X_test, y_trainval, y_test = train_test_split(X_aug, y_aug, test_size=0.10, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_trainval, y_trainval, test_size=0.20, random_state=42)

print("Train:", X_train.shape, y_train.shape)
print("Val: ", X_val.shape, y_val.shape)
print("Test: ", X_test.shape, y_test.shape)

np.save("X_train.npy", X_train)
np.save("y_train.npy", y_train)
np.save("X_val.npy", X_val)
np.save("y_val.npy", y_val)
np.save("X_test.npy", X_test)
np.save("y_test.npy", y_test)

BATCH_SIZE = 256

def make_ds(x, y, shuffle=True):
    ds = tf.data.Dataset.from_tensor_slices((x, y))

    if shuffle:
        ds = ds.shuffle(8910, reshuffle_each_iteration=True)

    ds = ds.batch(BATCH_SIZE)

```

```

ds = ds.prefetch(tf.data.AUTOTUNE)

# manually slicing targets into dict
def split_targets(a, b):
    # since we have 9 targets
    baron_targets = b[:, 0:3]    # y_baron_10,20,30
    dragon_targets = b[:, 3:6]   # y_dragon_10,20,30
    teamfight_targets = b[:, 6:9] # y_teamfight_10,20,30

    return a, {
        "baron": baron_targets,
        "dragon": dragon_targets,
        "teamfight": teamfight_targets
    }

ds = ds.map(split_targets)
return ds

train_ds = make_ds(X_train, y_train, shuffle=True)
val_ds = make_ds(X_val, y_val, shuffle=False)
Xtr_last = X_train[:, -1, :].astype(np.float32)
Xva_last = X_val[:, -1, :].astype(np.float32)
Xte_last = X_test[:, -1, :].astype(np.float32)

X_base = np.concatenate([Xtr_last, Xva_last], axis=0)
y_base = np.concatenate([y_train, y_val], axis=0)

print("Baseline training shapes:", X_base.shape, y_base.shape)

from sklearn.ensemble import RandomForestClassifier

def train_rf_per_label(X, y):
    rf_list = []
    for col_idx in range(y.shape[1]):
        rf = RandomForestClassifier(
            n_estimators=120,
            max_depth=5,
            min_samples_leaf=5,
            n_jobs=-1,
            class_weight="balanced_subsample",
            random_state=42,
        )
        rf.fit(X, y[:, col_idx])
        rf_list.append(rf)
    return rf_list

rf_models = train_rf_per_label(X_base, y_base)

def predict_rf(model_list, X):
    out_probs = []
    for model in model_list:

```

```

    p = model.predict_proba(X)[: , 1]
    out_probs.append(p)
    return np.stack(out_probs, axis=1)

y_test_pred_rf = predict_rf(rf_models, Xte_last)

print("RF test predictions shape:", y_test_pred_rf.shape)
event_horizons = [
    ("Baron", 10), ("Baron", 20), ("Baron", 30),
    ("Dragon", 10), ("Dragon", 20), ("Dragon", 30),
    ("Teamfight", 10), ("Teamfight", 20), ("Teamfight", 30)
]

event_names = [f"{e}_{h}" for e, h in event_horizons]
event_to_idx = {name: i for i, name in enumerate(event_names)}

print("Events:", event_to_idx)

rows = []

for name, idx in event_to_idx.items():
    yt = y_test[:, idx].astype(int)
    yp = y_test_pred_rf[:, idx]

    if yt.min() == yt.max():
        auc_val = np.nan
        brier_val = np.nan
    else:
        auc_val = roc_auc_score(yt, yp)
        brier_val = brier_score_loss(yt, yp)

    y_pred_bin = (yp >= 0.5).astype(int)
    prec_val = precision_score(yt, y_pred_bin, zero_division=0)

    event_name, horizon_val = name.split("_")
    horizon_val = int(horizon_val)

    rows.append({
        "event": event_name,
        "horizon": horizon_val,
        "auc": auc_val,
        "precision": prec_val,
        "brier": brier_val
    })

df_metrics_rf = pd.DataFrame(rows).sort_values(["event", "horizon"])

event_avg_rf = (
    df_metrics_rf.groupby("event")[["auc", "precision", "brier"]]
    .mean()
    .reset_index()
)

```

```

)

display(event_avg_rf)

def plot_main_conf_mats_rf():
    main_events = ["Baron_30", "Dragon_30", "Teamfight_30"]

    label_names = {
        "Baron": ("No Baron", "Baron"),
        "Dragon": ("No Dragon", "Dragon"),
        "Teamfight": ("No Teamfight", "Teamfight")
    }

    for ev_name in main_events:
        idx = event_to_idx[ev_name]
        y_true = y_test[:, idx].astype(int)
        y_prob = y_test_pred_rf[:, idx]
        y_pred = (y_prob >= 0.5).astype(int)
        cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
        base_name = ev_name.split("_")[0]
        lbl0, lbl1 = label_names[base_name]

        fig = px.imshow(
            cm,
            text_auto=True,
            x=[f"{lbl0}", f"{lbl1}"],
            y=[f"{lbl0}", f"{lbl1}"],
            labels=dict(
                x="Predicted",
                y="Actual",
                color="Count"
            ),
            title=base_name,
            color_continuous_scale="Blues"
        )

        fig.update_layout(width=350, height=350)
        fig.show()

plot_main_conf_mats_rf()

def plot_roc_30_rf():
    fig = go.Figure()
    main_events = ["Baron_30", "Dragon_30", "Teamfight_30"]

    for ev_name in main_events:
        idx = event_to_idx[ev_name]
        y_true = y_test[:, idx].astype(int)
        y_prob = y_test_pred_rf[:, idx]

        if y_true.min() == y_true.max():

```

continue

```
fpr, tpr, thresh = roc_curve(y_true, y_prob)
auc_val = roc_auc_score(y_true, y_prob)
clean_name = ev_name.split("_")[0]
legend_text = clean_name + " (AUC = " + format(auc_val, ".3f") + ")"
```

```
fig.add_trace(
    go.Scatter(
        x=fpr,
        y=tpr,
        mode="lines",
        name=legend_text
    )
)
```

```
fig.add_trace(
    go.Scatter(
        x=[0, 1],
        y=[0, 1],
        mode="lines",
        line=dict(dash="dash"),
        name="Random"
    )
)
```

```
fig.update_layout(
    title="ROC curves – Random Forest",
    xaxis_title="False Positive Rate",
    yaxis_title="True Positive Rate",
    xaxis=dict(range=[0, 1]),
    yaxis=dict(range=[0, 1]),
    width=750,
    height=550
)
```

```
fig.show()
```

```
plot_roc_30_rf()
```

```
def build_match_sequences(df, match_id, seq_len, step):
    g = df[df["match_id"] == match_id].sort_values("t").reset_index(drop=True)
    xf = g[feature_columns].values.astype(np.float32)
    yt = g[target_columns].values.astype(np.float32)

    xs, ys, ts = [], [], []
    for start_idx in range(0, len(g) - seq_len + 1, step):
        end_idx = start_idx + seq_len - 1
        xs.append(xf[start_idx:end_idx + 1])
        ys.append(yt[end_idx])
        ts.append(g.loc[end_idx, "t"])
```

```

if len(xs) == 0:
    return (
        np.empty((0, seq_len, len(feature_columns)), np.float32),
        np.empty((0, len(target_columns)), np.float32),
        np.empty((0,), np.float32)
    )
return np.array(xs), np.array(ys), np.array(ts, dtype=np.float32)

def baseline_seq_predict_rf(X_seq):
    X_last = X_seq[:, -1, :]
    probs9 = predict_rf(rf_models, X_last)

    return probs9[:, 0:3], probs9[:, 3:6], probs9[:, 6:9]

def pick_warnings(probs, t_sec, prob_floor, min_gap_s=45, max_warn=3):
    cand = np.where(probs >= prob_floor)[0]
    if len(cand) == 0:
        return np.array([], dtype=int)

    clusters = []
    cur = [cand[0]]

    for idx in cand[1:]:
        if t_sec[idx] - t_sec[cur[-1]] <= min_gap_s:
            cur.append(idx)
        else:
            clusters.append(cur)
            cur = [idx]
    clusters.append(cur)

    picks = []
    for c in clusters:
        c = np.array(c)
        picks.append(c[np.argmax(probs[c])])
    picks = np.array(picks)

    if len(picks) > max_warn:
        picks = picks[np.argsort(probs[picks])[:-1][:max_warn]]

    return picks

def plot_ews_dashboard_rf(match_id=None):
    if match_id is None:
        match_id = np.random.choice(df["match_id"].unique())
        print("Using match:", match_id)

    X_seq, Y_seq, T_seconds = build_match_sequences(df, match_id, SEQ_LEN, STEP)
    if X_seq.shape[0] == 0:
        print("No sequences for match", match_id)

```

```

return

T_minutes = T_seconds / 60.0
baron_p, dragon_p, teamfight_p = baseline_seq_predict_rf(X_seq)
cfg = {
    "Teamfight": dict(probs=teamfight_p[:, 2], y_col="y_teamfight_30",
                      color="red", prob_floor=0.25, max_warn=3),
    "Dragon": dict(probs=dragon_p[:, 2], y_col="y_dragon_30", color="green",
                  prob_floor=0.20, max_warn=2),
    "Baron": dict(probs=baron_p[:, 2], y_col="y_baron_30", color="blue",
                  prob_floor=0.12, max_warn=2)}

order = ["Teamfight", "Dragon", "Baron"]

fig = make_subplots(rows=3, cols=1, shared_xaxes=True, vertical_spacing=0.05,
                    subplot_titles=["Teamfight 30s risk (RF)", "Dragon 30s risk (RF)",
                                    "Baron 30s risk (RF)"])
fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="red"),
                        name="Teamfight risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="green"),
                        name="Dragon risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="blue"),
                        name="Baron risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="star",
                                size=13, color="gold",
                                line=dict(width=1, color="black")), name="Warning (hit)", row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="x", size=10,
                                color="firebrick"), name="Warning (false)", row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="square",
                                size=12, color="rgba(255,0,0,0.25)"), name="Actual event window", row=1, col=1)

for r, ev in enumerate(order, start=1):
    c = cfg[ev]
    probs = c["probs"]
    y_idx = target_columns.index(c["y_col"])
    y_true = Y_seq[:, y_idx].astype(int)

    picked = pick_warnings(probs, T_seconds, prob_floor=c["prob_floor"],
min_gap_s=45, max_warn=c["max_warn"])
    hit_idx = picked[y_true[picked] == 1]
    miss_idx = picked[y_true[picked] == 0]
    n_events = int(y_true.sum())
    n_warn = len(picked)
    n_hits = len(hit_idx)
    n_false = len(miss_idx)

    fig.add_trace(go.Scatter(x=T_minutes, y=probs, mode="lines+mark-
ers", line=dict(color=c["color"]), showlegend=False,
                    hovertemplate=(

```

```

f"{ev} risk<br>"
"Time %{x:.2f} min<br>"
"P(event in 30s)=%{y:.3f}<extra></extra>"
)), row=r, col=1)

fig.add_hline(y=c["prob_floor"], row=r, col=1, line_dash="dash", line_color=c["color"], opacity=0.5)

if len(hit_idx):
    fig.add_trace(go.Scatter(x=T_minutes[hit_idx], y=probs[hit_idx], mode="markers",
marker=dict(symbol="star", size=13,
color="gold", line=dict(width=1, color="black")), showlegend=False), row=r, col=1)

if len(miss_idx):
    fig.add_trace(go.Scatter(x=T_minutes[miss_idx], y=probs[miss_idx], mode="markers",
marker=dict(symbol="x", size=10,
color="firebrick"), showlegend=False), row=r, col=1)

ev_times = np.unique(T_seconds[y_true == 1])
for t in ev_times:
    fig.add_vrect(x0=t / 60.0, x1=(t + 30.0) / 60.0, row=r, col=1, fillcolor="red", opacity=0.12,
line_width=1, line_dash="dot")

summary = (f"Events: {n_events} | " f"Warnings: {n_warn} | " f"Hits: {n_hits} | " f"False:
{n_false}")
fig.add_annotation(text=summary, xref="x domain", yref="y domain", x=0.99, y=0.96, xanchor="right",
yanchor="top", showarrow=False, font=dict(size=10), row=r, col=1)

if r == 2:
    fig.update_yaxes(title_text="P(event in 30s)", row=2, col=1)

fig.update_xaxes(title_text="Game time (min)", row=3, col=1)
fig.update_layout(height=760, title=f"Random Forest – risk plot: {match_id}", hovermode="x unified",
margin=dict(t=80, r=10, l=70, b=60),
legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1))

fig.show()

plot_ews_dashboard_rf("EUW1_7497397957")
# flattening the time dimension
Xtr_last = X_train[:, -1, :].astype(np.float32)
Xva_last = X_val[:, -1, :].astype(np.float32)
Xte_last = X_test[:, -1, :].astype(np.float32)

# combine train + val
X_base = np.concatenate([Xtr_last, Xva_last], axis=0)
y_base = np.concatenate([y_train, y_val], axis=0)

import tensorflow as tf
from tensorflow.keras import layers, models

```



```

def build_simple_nn(input_dim, output_dim=9):
    inp = layers.Input(shape=(input_dim,))
    x = layers.Dense(64, activation="relu")(inp)
    x = layers.Dense(32, activation="relu")(x)
    out = layers.Dense(output_dim, activation="sigmoid")(x)
    model = models.Model(inp, out)
    return model

simple_nn = build_simple_nn(X_base.shape[1])
simple_nn.summary()

simple_nn.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3), loss="binary_crossentropy", metrics=["AUC"])
history_nn = simple_nn.fit(X_base, y_base, epochs=12, batch_size=256, validation_split=0.1, verbose=1)
y_test_pred = simple_nn.predict(Xte_last)
event_horizons = [
    ("Baron", 10), ("Baron", 20), ("Baron", 30),
    ("Dragon", 10), ("Dragon", 20), ("Dragon", 30),
    ("Teamfight", 10), ("Teamfight", 20), ("Teamfight", 30)
]

event_names = []
for e, h in event_horizons:
    event_names.append(e + "_" + str(h))

event_to_idx = {}
for i, name in enumerate(event_names):
    event_to_idx[name] = i
rows = []

for name, idx in event_to_idx.items():
    yt = y_test[:, idx].astype(int)
    yp = y_test_pred[:, idx]

    if yt.min() == yt.max():
        auc_val = np.nan
        brier_val = np.nan
    else:
        auc_val = roc_auc_score(yt, yp)
        brier_val = brier_score_loss(yt, yp)

    y_bin = (yp >= 0.5).astype(int)
    prec_val = precision_score(yt, y_bin, zero_division=0)
    tmp = name.split("_")
    base_name = tmp[0]
    the_horizon = int(tmp[1])

    row = {
        "event": base_name,

```

```

        "horizon": the_horizon,
        "auc": auc_val,
        "precision": prec_val,
        "brier": brier_val
    }
    rows.append(row)

df_metrics = pd.DataFrame(rows)
df_metrics = df_metrics.sort_values(["event", "horizon"])
event_avg = (df_metrics.groupby("event")[["auc", "precision", "brier"]].mean().reset_index())
display(event_avg)
def plot_main_conf_mats_baseline():
    main_events = ["Baron_30", "Dragon_30", "Teamfight_30"]
    map_labels = {"Baron": ("No Baron", "Baron"),
                  "Dragon": ("No Dragon", "Dragon"),
                  "Teamfight": ("No Teamfight", "Teamfight")}

    for ev_name in main_events:
        idx = event_to_idx[ev_name]
        yt = y_test[:, idx].astype(int)
        yp = y_test_pred[:, idx]
        yb = (yp >= 0.5).astype(int)
        cm = confusion_matrix(yt, yb, labels=[0, 1])
        base = ev_name.split("_")[0]
        true0, true1 = map_labels[base]
        fig = px.imshow(cm, text_auto=True, x=[f"{true0}", f"{true1}"], y=[f"{true0}", f"{true1}"],
                        labels=dict(x="Predicted", y="Actual", color="Count"), title=base,
                        color_continuous_scale="Blues")
        fig.update_layout(width=350, height=350)
        fig.show()
    plot_main_conf_mats_baseline()
def plot_roc_30_baseline():
    fig = go.Figure()
    the_events = ["Baron_30", "Dragon_30", "Teamfight_30"]

    for ev in the_events:
        idx = event_to_idx[ev]
        y_true = y_test[:, idx].astype(int)
        y_prob = y_test_pred[:, idx]
        if y_true.min() == y_true.max():
            continue

        fpr, tpr, thr = roc_curve(y_true, y_prob)
        auc_val = roc_auc_score(y_true, y_prob)
        label_name = ev.split("_")[0]
        leg_text = label_name + " (AUC = " + format(auc_val, ".3f") + ")"
        fig.add_trace(go.Scatter(x=fpr, y=tpr, mode="lines", name=leg_text))

    fig.add_trace(go.Scatter(x=[0, 1], y=[0, 1], mode="lines", line=dict(dash="dash"), name="Random"))
    fig.update_layout(title="ROC curve - Simple MLP", xaxis_title="False Positive Rate", yaxis_title="True Positive Rate",

```

```

        xaxis=dict(range=[0, 1]), yaxis=dict(range=[0, 1]), width=750, height=550)
    fig.show()
plot_roc_30_baseline()
import plotly.graph_objects as go
from plotly.subplots import make_subplots

def plot_simple_nn_training(history_nn):
    hist = history_nn.history
    train_loss = hist["loss"]
    val_loss = hist["val_loss"]
    train_auc = hist["AUC"]
    val_auc = hist["val_AUC"]
    epochs = list(range(1, len(train_loss) + 1))
    fig = make_subplots(rows=1, cols=2, subplot_titles=("AUC", "Loss"))
    fig.add_trace(go.Scatter(x=epochs, y=train_auc, mode="lines+markers", name="Train AUC",
line=dict(color="royalblue")), row=1, col=1)
    fig.add_trace(go.Scatter(x=epochs, y=val_auc, mode="lines+markers", name="Val AUC",
line=dict(color="firebrick")), row=1, col=1)
    fig.add_trace(go.Scatter(x=epochs, y=train_loss, mode="lines+markers", name="Train Loss",
line=dict(color="green")), row=1, col=2)
    fig.add_trace(go.Scatter(x=epochs, y=val_loss, mode="lines+markers", name="Val Loss",
line=dict(color="orange")), row=1, col=2)
    fig.update_xaxes(title_text="Epoch")
    fig.update_yaxes(title_text="AUC", row=1, col=1)
    fig.update_yaxes(title_text="Loss", row=1, col=2)
    fig.update_layout(title="Simple MLP — Epoch plot", height=450, width=1000, legend=dict(orienta-
tion="h",
        yanchor="bottom", y=-0.25, xanchor="center", x=0.5))

    fig.show()
plot_simple_nn_training(history_nn)
def build_match_sequences(df, match_id, seq_len, step):
    g = df[df["match_id"] == match_id].sort_values("t").reset_index(drop=True)
    xf = g[feature_columns].values.astype(np.float32)
    yt = g[target_columns].values.astype(np.float32)

    xs, ys, ts = [], [], []
    for start_idx in range(0, len(g) - seq_len + 1, step):
        end_idx = start_idx + seq_len - 1
        xs.append(xf[start_idx:end_idx + 1])
        ys.append(yt[end_idx])
        ts.append(g.loc[end_idx, "t"])

    if len(xs) == 0:
        return (
            np.empty((0, seq_len, len(feature_columns)), np.float32),
            np.empty((0, len(target_columns)), np.float32),
            np.empty((0,), np.float32)
        )

    return np.array(xs), np.array(ys), np.array(ts, dtype=np.float32)

```

```

def pick_warnings(probabilities, t_sec, prob_floor, min_gap_s=45, max_warn=3):
    cand = np.where(probabilities >= prob_floor)[0]
    if len(cand) == 0:
        return np.array([], dtype=int)

    clusters = []
    cur = [cand[0]]
    for idx in cand[1:]:
        if t_sec[idx] - t_sec[cur[-1]] <= min_gap_s:
            cur.append(idx)
        else:
            clusters.append(cur)
            cur = [idx]
    clusters.append(cur)

    picks = []
    for c in clusters:
        c = np.array(c)
        picks.append(c[np.argmax(probabilities[c])])
    picks = np.array(picks)

    if len(picks) > max_warn:
        picks = picks[np.argsort(probabilities[picks])[:-1][:max_warn]]

    return picks

def predict_baseline_nn(X):
    return simple_nn.predict(X, verbose=0)

def baseline_seq_predict(X_seq):
    X_last = X_seq[:, -1, :]
    probs_all = predict_baseline_nn(X_last)

    baron_p = probs_all[:, 0:3]
    dragon_p = probs_all[:, 3:6]
    teamfight_p = probs_all[:, 6:9]

    return baron_p, dragon_p, teamfight_p

def plot_ews_dashboard_baseline(match_id=None):
    if match_id is None:
        match_id = np.random.choice(df["match_id"].unique())
    print("Using match:", match_id)

    X_seq, Y_seq, T_sec = build_match_sequences(df, match_id, SEQ_LEN, STEP)
    if X_seq.shape[0] == 0:
        print("No sequences for match", match_id)
        return

    T_min = T_sec / 60.0

```

```

b_p, d_p, t_p = baseline_seq_predict(X_seq)
cfg = {
    "Teamfight": dict(
        probs=t_p[:, 2],
        y_col="y_teamfight_30",
        color="red",
        prob_floor=0.25,
        max_warn=3,
    ),
    "Dragon": dict(
        probs=d_p[:, 2],
        y_col="y_dragon_30",
        color="green",
        prob_floor=0.20,
        max_warn=2,
    ),
    "Baron": dict(
        probs=b_p[:, 2],
        y_col="y_baron_30",
        color="blue",
        prob_floor=0.12,
        max_warn=2,
    ),
}

order = ["Teamfight", "Dragon", "Baron"]
fig = make_subplots(rows=3, cols=1, shared_xaxes=True, vertical_spacing=0.05,
    subplot_titles=[
        "Teamfight 30s risk (Simple NN)",
        "Dragon 30s risk (Simple NN)",
        "Baron 30s risk (Simple NN)"
    ]
)

fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="red"), name="Team-
fight risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="green"),
name="Dragon risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="blue"), name="Baron
risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="star",
size=13, color="gold",
    line=dict(width=1, color="black")), name="Warning (hit)", row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="x", size=10,
color="firebrick"), name="Warning (false)", row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="square",
size=12, color="rgba(255,0,0,0.25)"), name="Actual event window", row=1, col=1)

for r, ev in enumerate(order, start=1):
    c = cfg[ev]
    probs = c["probs"]

```

```

y_idx = target_columns.index(c["y_col"])
y_true = Y_seq[:, y_idx].astype(int)
picked = pick_warnings(probs, T_sec, prob_floor=c["prob_floor"], min_gap_s=45,
max_warn=c["max_warn"])
hit_idx = picked[y_true[picked] == 1]
miss_idx = picked[y_true[picked] == 0]
n_events = int(y_true.sum())
n_warn = len(picked)
n_hits = len(hit_idx)
n_false = len(miss_idx)
fig.add_trace(go.Scatter(x=T_min, y=probs, mode="lines+markers", line=dict(color=c["color"]),
showlegend=False,
    hovertemplate=(
        f"{ev} risk<br>"
        "Time %{x:.2f} min<br>"
        "P(event in 30s)=%{y:.3f}<extra></extra>"
    )
), row=r, col=1)

fig.add_hline(y=c["prob_floor"], line_dash="dash", line_color=c["color"], opacity=0.5, row=r,
col=1)

if len(hit_idx):
    fig.add_trace(go.Scatter(x=T_min[hit_idx], y=probs[hit_idx], mode="mark-
ers", marker=dict(symbol="star", size=13,
        color="gold", line=dict(width=1, color="black")), showlegend=False), row=r, col=1)

if len(miss_idx):
    fig.add_trace(go.Scatter(x=T_min[miss_idx], y=probs[miss_idx], mode="markers",
marker=dict(symbol="x", size=10,
        color="firebrick"), showlegend=False), row=r, col=1)

ev_times = np.unique(T_sec[y_true == 1])
for t in ev_times:
    fig.add_vrect(x0=t/60.0, x1=(t+30)/60.0, fillcolor="red", opacity=0.12, line_width=1,
line_dash="dot", row=r, col=1)

summary = (f"Events: {n_events} | " f"Warnings: {n_warn} | " f"Hits: {n_hits} | " f"False:
{n_false}" )
fig.add_annotation(text=summary, xref="x domain", yref="y domain", x=0.99, y=0.96, xan-
chor="right", yanchor="top",
    showarrow=False, font=dict(size=10), row=r, col=1)

if r == 2:
    fig.update_yaxes(title_text="P(event in 30s)")

fig.update_xaxes(title_text="Game time (min)", row=3, col=1)
fig.update_layout(height=760, title=f"Simple MLP – risk plot: {match_id}", hovermode="x unified",
margin=dict(t=80, r=10, l=70, b=60),
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1))
fig.show()

```

```

plot_ews_dashboard_baseline("EUW1_7497397957")
import keras_tuner as kt

def squeeze_excite(x, r=8):
    f = x.shape[-1]
    se = layers.GlobalAveragePooling1D()(x)
    se = layers.Dense(int(f // r), activation="relu")(se)
    se = layers.Dense(int(f), activation="sigmoid")(se)
    se = layers.Reshape((1, int(f)))(se)
    out = layers.Multiply()(x, se)
    return out

def res_tcn(x, f, k=3, d=1, drop=0.15):
    y = layers.Conv1D(filters=f, kernel_size=k, dilation_rate=d, padding="causal", activation="relu")(x)
    y = layers.BatchNormalization()(y)
    y = layers.Dropout(drop)(y)
    y = layers.Conv1D(filters=f, kernel_size=k, dilation_rate=d, padding="causal")(y)
    y = layers.BatchNormalization()(y)
    y = squeeze_excite(y)
    if x.shape[-1] != f:
        x = layers.Conv1D(f, 1, padding="same")(x)
    out = layers.Add()(x, y)
    out = layers.Activation("relu")(out)
    return out

def cross_att(x1, x2, h=4, d=64):
    att_layer = layers.MultiHeadAttention(num_heads=h, key_dim=d)
    got = att_layer(x1, x2)
    skip = layers.Add()(x1, got)
    out = layers.LayerNormalization()(skip)
    return out

def build_LeagueEWS(hp, seq_len=SEQ_LEN, n_feat=len(feature_columns)):
    tcn_filters = hp.Choice("tcn_filters", values=[48, 64, 80])
    gru_units = hp.Choice("gru_units", values=[96, 128, 160])
    dropout_rate = hp.Choice("shared_dropout", values=[0.3, 0.4, 0.5])
    lr = hp.Choice("learning_rate", values=[5e-5, 7e-5, 1e-4])

    inp = layers.Input(shape=(seq_len, n_feat))
    t = res_tcn(inp, tcn_filters)
    t = res_tcn(t, tcn_filters * 2, k=3, d=2)
    t = res_tcn(t, tcn_filters * 2, k=3, d=4)
    t = layers.TimeDistributed(layers.Dense(128, activation="relu"))(t)
    g = layers.Bidirectional(layers.GRU(gru_units, return_sequences=True, dropout=0.3))(inp)
    g = layers.Bidirectional(layers.GRU(gru_units, return_sequences=True, dropout=0.3))(g)
    xx = cross_att(t, g)
    xx = cross_att(xx, inp)
    attn = layers.MultiHeadAttention(4, 64)(xx, xx)
    xx = layers.Add()(xx, attn)
    xx = layers.LayerNormalization()(xx)

```

```

ff = layers.Dense(256, activation="relu")(xx)
ff = layers.Dense(xx.shape[-1])(ff)
xx = layers.Add()([xx, ff])
w = layers.Dense(1, activation="tanh")(xx)
w = layers.Softmax(axis=1)(w)
pooled = layers.Multiply()([xx, w])
pooled = layers.Lambda(lambda z: tf.reduce_sum(z, axis=1))(pooled)
h = layers.Dense(512, activation="relu", kernel_regularizer=regularizers.l2(1e-6))(pooled)
h = layers.Dropout(dropout_rate)(h)
shared = layers.Dense(256, activation="relu")(h)

baron_out = layers.Dense(3, activation="sigmoid", name="baron")(layers.Dropout(0.5)(shared))
dragon_out = layers.Dense(3, activation="sigmoid", name="dragon")(shared)
fight_out = layers.Dense(3, activation="sigmoid", name="teamfight")(shared)

model = models.Model(inputs=inp, outputs=[baron_out, dragon_out, fight_out])
opt = tf.keras.optimizers.AdamW(
    learning_rate=lr,
    weight_decay=0.0001
)

model.compile(
    optimizer = opt,
    loss = {
        "baron": "binary_crossentropy",
        "dragon": "binary_crossentropy",
        "teamfight": "binary_crossentropy"
    },
    loss_weights = {
        "baron": 1.0,
        "dragon": 2.0,
        "teamfight": 2.5
    },
    metrics = {
        "baron": [AUC(name="baron_auc")],
        "dragon": [AUC(name="dragon_auc")],
        "teamfight": [AUC(name="teamfight_auc")]
    }
)
return model
import keras_tuner as kt
import numpy as np
import tensorflow as tf

class MeanAUC(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        if logs is None:
            return

        b = logs.get("val_baron_baron_auc")
        d = logs.get("val_dragon_dragon_auc")

```



```

t = logs.get("val_teamfight_teamfight_auc")
valid = [v for v in [b, d, t] if v is not None]

if len(valid) == 0:
    logs["val_mean_auc"] = np.nan
else:
    logs["val_mean_auc"] = float(np.nanmean(valid))

print(f" → val_mean_auc: {logs['val_mean_auc']:.4f}")

tuner = kt.RandomSearch(hypermodel=build_LeagueEWS, objective=kt.Objective("val_mean_auc",
direction="max"), max_trials=5,
                        executions_per_trial=1, directory="LeagueEWS_tuning", project_name="ews_hp")
tuner.search(train_ds, validation_data=val_ds, epochs=6, callbacks=[MeanAUC()], verbose=1)
best_hp = tuner.get_best_hyperparameters(1)[0]
print("Best hyperparameters found:", best_hp.values)
results = tuner.oracle.get_best_trials(num_trials=20)
trial_summaries = []

for t in results:
    trial_summaries.append({"trial_id": t.trial_id, "val_mean_auc": t.score, **t.hyperparameters.values})

df_tuner = pd.DataFrame(trial_summaries)
display(df_tuner)
model = build_LeagueEWS(best_hp)
model.summary()
EPOCHS = 30
steps_per_epoch = int(np.ceil(len(X_train) / BATCH_SIZE))
total_steps = steps_per_epoch * EPOCHS
initial_lr = 1e-4
lr_schedule = tf.keras.optimizers.schedules.CosineDecay(initial_learning_rate = initial_lr, decay_steps
= total_steps, alpha = 0.2)

if isinstance(model.optimizer.learning_rate, float) or isinstance(model.optimizer.learning_rate,
tf.Variable):
    model.optimizer.learning_rate = lr_schedule

class MinEpochEarlyStopping(tf.keras.callbacks.Callback):
    def __init__(self, monitor="val_loss", min_epochs=12, patience=6, mode="min"):
        super(MinEpochEarlyStopping, self).__init__()
        self.monitor = monitor
        self.min_epochs = min_epochs
        self.patience = patience
        self.mode = mode
        self.best = None
        self.wait = 0

    def on_epoch_end(self, epoch, logs=None):
        if logs is None:
            logs = {}

```

```

cur_value = logs.get(self.monitor)
if cur_value is None:
    return

if self.best is None:
    self.best = cur_value

if epoch < (self.min_epochs - 1):
    if (self.mode == "min" and cur_value < self.best) or \
        (self.mode == "max" and cur_value > self.best):
        self.best = cur_value
    return

improved = ((self.mode == "min" and cur_value < self.best) or (self.mode == "max" and
cur_value > self.best))

if improved:
    self.best = cur_value
    self.wait = 0
else:
    self.wait += 1
    if self.wait >= self.patience:
        print(f"\nEarly Stopping at epoch {epoch+1}. Best {self.monitor} = {self.best:.4f}")
        self.model.stop_training = True

checkpoint_path = "LeagueEWS.keras"
callbacks_list = [MinEpochEarlyStopping(monitor = "val_loss", min_epochs = 12, patience = 6, mode =
"min"),
                  tf.keras.callbacks.ModelCheckpoint(filepath = checkpoint_path, monitor = "val_loss",
                                                         save_best_only = True, verbose = 1)]

history = model.fit(train_ds, validation_data = val_ds, epochs = EPOCHS, callbacks = callbacks_list,
verbose=1)
model.load_weights(checkpoint_path)
def stack_out(o):
    if isinstance(o, list) or isinstance(o, tuple):
        return np.concatenate(o, axis=-1)
    else:
        return o

y_test_pred = model.predict(X_test, batch_size=512, verbose=0)
y_test_pred = stack_out(y_test_pred)

event_horizons = [("Baron", 10), ("Baron", 20), ("Baron", 30),
                  ("Dragon", 10), ("Dragon", 20), ("Dragon", 30),
                  ("Teamfight", 10), ("Teamfight", 20), ("Teamfight", 30),
                  ]

event_names = []
for e, h in event_horizons:
    event_names.append(f"{e}_{h}")

```

```

event_to_idx = {}
for i, nm in enumerate(event_names):
    event_to_idx[nm] = i

rows = []
for name, idx in event_to_idx.items():
    yt = y_test[:, idx].astype(int)
    yp = y_test_pred[:, idx]

    try:
        auc_val = roc_auc_score(yt, yp)
        brier_val = brier_score_loss(yt, yp)
    except ValueError:
        auc_val = np.nan
        brier_val = np.nan

    y_bin = (yp >= 0.5).astype(int)
    prec_val = precision_score(yt, y_bin, zero_division=0)
    base_name, horizon_str = name.split("_")
    horizon_num = int(horizon_str)

    rows.append({
        "event": base_name,
        "horizon": horizon_num,
        "auc": auc_val,
        "precision": prec_val,
        "brier": brier_val
    })

df_metrics = pd.DataFrame(rows)
df_metrics = df_metrics.sort_values(["event", "horizon"])

event_avg = (df_metrics.groupby("event")[["auc", "precision", "brier"]].mean().reset_index())
display(event_avg)
def plot_main_conf_mats():
    label_map = {
        "Baron": ["No Baron", "Baron"],
        "Dragon": ["No Dragon", "Dragon"],
        "Teamfight": ["No Teamfight", "Teamfight"]
    }

    main = ["Baron_30", "Dragon_30", "Teamfight_30"]

    for event_name in main:
        idx = event_to_idx[event_name]
        yt = y_test[:, idx].astype(int)
        yp = y_test_pred[:, idx]
        y_pred_bin = (yp >= 0.5).astype(int)
        cm = confusion_matrix(yt, y_pred_bin, labels=[0, 1])
        base_event = event_name.split("_")[0]

```

```

true_labels = label_map[base_event]
pred_labels = label_map[base_event]
fig = px.imshow(cm, text_auto=True, x=[f"Pred: {pred_labels[0]}" , f"Pred: {pred_labels[1]}"],
y=[f"True: {true_labels[0]}" , f"True: {true_labels[1]}"],
labels=dict(x="Predicted", y="True", color="Count"), title=f"{base_event} – Confusion
Matrix", color_continuous_scale="Blues")
fig.update_layout(width=350, height=350)
fig.show()
plot_main_conf_mats()
import plotly.graph_objects as go
from plotly.subplots import make_subplots

def plot_event_training_curves(history):
    hist = history.history

    events = [
        ("Baron", "baron_baron_auc", "baron_loss", "val_baron_baron_auc", "val_baron_loss"),
        ("Dragon", "dragon_dragon_auc", "dragon_loss", "val_dragon_dragon_auc", "val_dragon_loss"),
        ("Teamfight", "teamfight_teamfight_auc", "teamfight_loss", "val_teamfight_teamfight_auc",
"val_teamfight_loss")]

    fig = make_subplots(rows=3, cols=2,
        subplot_titles=[
            "Baron AUC", "Baron Loss",
            "Dragon AUC", "Dragon Loss",
            "Teamfight AUC", "Teamfight Loss"
        ], vertical_spacing=0.12, horizontal_spacing=0.12)

    legend_added = {
        "train_auc": False,
        "val_auc": False,
        "train_loss": False,
        "val_loss": False
    }

    for r, (ev, auc_t, loss_t, auc_v, loss_v) in enumerate(events, start=1):
        fig.add_trace(go.Scatter(y=hist[auc_t], mode="lines+markers", line=dict(color="royalblue"),
name="Train AUC" if not legend_added["train_auc"] else None,
        showlegend=not legend_added["train_auc"]), row=r, col=1)
        legend_added["train_auc"] = True
        fig.add_trace(go.Scatter(y=hist[auc_v], mode="lines+markers", line=dict(color="firebrick"),
name="Val AUC" if not legend_added["val_auc"] else None,
        showlegend=not legend_added["val_auc"]), row=r, col=1)
        legend_added["val_auc"] = True
        fig.add_trace(go.Scatter(y=hist[loss_t], mode="lines+markers", line=dict(color="green"),
name="Train Loss" if not legend_added["train_loss"] else None,
        showlegend=not legend_added["train_loss"]), row=r, col=2)
        legend_added["train_loss"] = True
        fig.add_trace(go.Scatter(y=hist[loss_v], mode="lines+markers", line=dict(color="orange"),
name="Val Loss" if not legend_added["val_loss"] else None,
        showlegend=not legend_added["val_loss"]), row=r, col=2)

```

```

legend_added["val_loss"] = True

fig.update_layout(height=1400, width=1000, title="LeagueEWS - Epoch plot", showlegend=True)
fig.update_xaxes(title_text="Epoch")
fig.show()

plot_event_training_curves(history)
def plot_roc_30():
    fig = go.Figure()
    events_list = ["Baron_30", "Dragon_30", "Teamfight_30"]
    for ev in events_list:
        idx = event_to_idx[ev]
        yt = y_test[:, idx].astype(int)
        yp = y_test_pred[:, idx]
        if yt.min() == yt.max():
            continue

        fpr, tpr, whatever_unused = roc_curve(yt, yp)
        auc_val = roc_auc_score(yt, yp)
        short_name = ev.split("_")[0]
        legend_label = short_name + " (AUC = " + format(auc_val, ".3f") + ")"
        fig.add_trace(go.Scatter(x=fpr, y=tpr, mode="lines", name=legend_label))

    fig.add_trace(go.Scatter(x=[0, 1], y=[0, 1], mode="lines", line=dict(dash="dash"), showlegend=True,
name="Random"))
    fig.update_layout(
        title="ROC curves – LeagueEWS",
        xaxis_title="False Positive Rate",
        yaxis_title="True Positive Rate",
        xaxis=dict(range=[0, 1]),
        yaxis=dict(range=[0, 1]),
        width=750,
        height=550
    )

    fig.show()

plot_roc_30()
def plot_ews_dashboard(match_id=None):
    if match_id is None:
        all_ids = df["match_id"].unique()
        match_id = np.random.choice(all_ids)
    print("Using match:", match_id)

    X_seq, Y_seq, T_seconds = build_match_sequences(df, match_id, SEQ_LEN, STEP)
    if X_seq.shape[0] == 0:
        print("No sequences for match", match_id)
        return

    T_minutes = T_seconds / 60.0
    preds = model.predict(X_seq, verbose=0)

```

```

baron_probs = preds[0]
dragon_probs = preds[1]
teamfight_probs = preds[2]
cfg = {
    "Teamfight": {
        "probs": teamfight_probs[:, 2],
        "y_col": "y_teamfight_30",
        "color": "red",
        "prob_floor": 0.25,
        "max_warn": 3
    },
    "Dragon": {
        "probs": dragon_probs[:, 2],
        "y_col": "y_dragon_30",
        "color": "green",
        "prob_floor": 0.20,
        "max_warn": 2
    },
    "Baron": {
        "probs": baron_probs[:, 2],
        "y_col": "y_baron_30",
        "color": "blue",
        "prob_floor": 0.12,
        "max_warn": 2
    }
}

```

```

event_order = ["Teamfight", "Dragon", "Baron"]
fig = make_subplots(rows=3, cols=1, shared_xaxes=True, vertical_spacing=0.05,
    subplot_titles=[
        "Teamfight 30s risk",
        "Dragon 30s risk",
        "Baron 30s risk"
    ]
)

```

```

fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="red"), name="Team-
fight risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="green"),
name="Dragon risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="lines", line=dict(color="blue"), name="Baron
risk"), row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="star",
size=12, color="gold", line=dict(width=1, color="black")),
    name="Warning (hit)", row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="x", size=10,
color="firebrick"), name="Warning (false)", row=1, col=1)
fig.add_trace(go.Scatter(x=[None], y=[None], mode="markers", marker=dict(symbol="square",
size=12, color="rgba(255,0,0,0.25)"), name="Actual event window"), row=1, col=1)

```

```

for r, ev_name in enumerate(event_order, start=1):

```

```

stuff = cfg[ev_name]
probs = stuff["probs"]
target_idx = target_columns.index(stuff["y_col"])
y_true = Y_seq[:, target_idx].astype(int)
warn_idx = pick_warnings(probs, T_seconds, prob_floor=stuff["prob_floor"], min_gap_s=45,
max_warn=stuff["max_warn"])
hit_idx = warn_idx[y_true[warn_idx] == 1]
miss_idx = warn_idx[y_true[warn_idx] == 0]
n_events = int(y_true.sum())
n_warn = len(warn_idx)
n_hits = len(hit_idx)
n_false = len(miss_idx)

fig.add_trace(go.Scatter(x=T_minutes, y=probs, mode="lines+markers",
line=dict(color=stuff["color"]), showlegend=False, name=ev_name), row=r, col=1)
fig.add_hline(y=stuff["prob_floor"], row=r, col=1, line_dash="dash", line_color=stuff["color"],
opacity=0.5)

if len(hit_idx) > 0:
    fig.add_trace(go.Scatter(x=T_minutes[hit_idx], y=probs[hit_idx], mode="markers",
marker=dict(symbol="star", size=13, color="gold",
line=dict(width=1, color="black")), showlegend=False), row=r, col=1)

if len(miss_idx) > 0:
    fig.add_trace(go.Scatter(x=T_minutes[miss_idx], y=probs[miss_idx], mode="markers",
marker=dict(symbol="x", size=10, color="firebrick"), showlegend=False), row=r, col=1)

ev_times = np.unique(T_seconds[y_true == 1])
for t in ev_times:
    fig.add_vrect(x0=t/60.0, x1=(t + 30.0)/60.0, row=r, col=1, fillcolor="red", opacity=0.12,
line_width=1, line_dash="dot")

summary_text = f"Events: {n_events} | Warnings: {n_warn} | Hits: {n_hits} | False: {n_false}"
fig.add_annotation(text=summary_text, xref="x domain", yref="y domain", x=0.99, y=0.96, xan-
chor="right", yanchor="top", showarrow=False, font=dict(size=10), row=r, col=1)

if r == 2:
    fig.update_yaxes(title_text="P(event in 30s)", row=2, col=1, range=[0,1])

fig.update_xaxes(title_text="Game time (min)", row=3, col=1)
fig.update_layout(height=740, title="League EWS – risk plot:" + str(match_id), hovermode="x uni-
fied", margin=dict(t=80, r=10, l=70, b=60),
legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1))
fig.show()
plot_ews_dashboard("EUW1_7497397957")
plot_ews_dashboard("EUW1_7564257918")
plot_ews_dashboard("EUW1_7579203833")
plot_ews_dashboard("EUW1_7584511607")

```